

Akshar Devanagari IME

Source Manual — Architecture, Mathematics, Training and Evaluation

Akshar IME

22 September 2026

Contents

1	About this manual	4
1.1	Conventions	4
1.2	Reproducing everything in this manual	4
2	What this system is	6
2.1	Contribution and prior art	6
2.2	What it is not	7
2.3	Measured performance	7
3	Quick start	9
3.1	Build and install (Linux / IBus)	9
3.2	Use from Rust	9
3.3	Use from the browser	9
4	Problem formulation	10
4.1	The task	10
4.2	The source-channel decomposition	10
4.3	The unit of modelling: the akshara	11
4.4	Alignment is latent	11
5	The transliteration model and EM training	12
5.1	Parameters	12
5.2	Initialisation	12
5.3	The E-step: scaled forward-backward	12
5.3.1	Why the passes are scaled	13
5.4	The M-step	13
5.5	What the model looks like when trained	14
6	The language model	15
6.1	Why Kneser-Ney	15
6.2	Modified Kneser-Ney discounts	15
6.3	The interpolated model	16
6.4	Storage and lookup	16
6.5	Two historical deviations from textbook KN (resolved in v1.2.0)	16
7	Decoding	17
7.1	The lattice	17
7.2	Beam search	17
7.3	Two passes: free and dictionary-constrained	18
7.4	What the decoder returns	18

8 Discriminative reranking	19
8.1 The baseline heuristic	19
8.2 Dense features	19
8.3 Sparse lexicalized features	20
8.4 The blend	20
8.5 The cascade	21
8.6 Language-conditioned ranking and the shared lexicon	21
8.7 Does the shared automaton transfer, or just compact? (ablation)	22
9 The engine: candidate fusion and learning	24
9.1 Sources of evidence	24
9.2 Digits and punctuation	24
9.3 Adaptive learning	25
9.3.1 How fuzzy matching actually behaves	25
10 The model container	26
10.1 One file	26
10.2 Versions	26
10.3 Compact encoding	27
10.4 Browser profile	27
11 Training	28
11.1 What --reranker-pairs does and does not control	28
11.2 The four phases	28
11.3 Learning rate	29
11.4 Held-out monitoring	29
11.5 Data	29
12 Evaluation methodology	30
12.1 Data	30
12.2 Metrics	31
12.3 Protocol	32
12.4 Statistical treatment	33
12.5 Threats to validity	33
12.6 Harnesses	34
12.7 The accuracy regression guard	34
12.8 Headroom, and where the errors are	34
12.9 The collision bound	35
12.10 Cold vs. session accuracy	35
13 Ablations: what each component is worth	37
13.1 Switches	37
13.2 The rerank stage, built up from raw decoder order	37
13.3 Contribution of each remaining component	38
13.4 Paired significance	38
13.5 Components that were removed	39
13.6 Modified Kneser-Ney against a single discount	39
14 Performance	41
14.1 Where the time goes	41
14.2 How it got there	41
14.3 Trade-offs available	42

15 Deployment	43
15.1 Linux / IBus	43
15.2 Browser / WebAssembly	43
15.3 Offline tools	44
16 Source map	45
17 Known defects and limitations	47
17.1 Defects resolved in v1.2.0	47
17.2 Open limitations	48
17.3 Things that do not work as their names suggest	48
17.4 Scope limitations	48
18 Roadmap	50
18.1 Result: more supervision was tested and does not help	50
18.2 Resolution: Joint Dense + Sparse Training (Landed in v1.2.0)	51
18.2.1 Measured Results on train-mid and train-full (v1.2.0)	51
18.2.2 Falsification Test Finding	52
19 Experimental record	53
19.1 Approaches evaluated and rejected	53
19.2 Approaches considered but not implemented	53
19.3 Archive index	54
20 References	55
20.1 Model and training	55
20.2 Language modelling	55
20.3 Decoding	56
20.4 Discriminative reranking	56
20.5 Strings, structures and statistics	56
20.6 Data, benchmarks and writing systems	57
20.7 Implementation	57
21 Glossary	58

Chapter 1

About this manual

This is the complete reference for Akshar Devanagari IME: what it computes, how it is trained, how it is measured, and how to reproduce every number in it. It is written to be read start to finish by someone who has never seen the codebase.

Every quantitative claim here was measured on the tree it describes. Where a figure has not been re-measured since a change, the text says so rather than carrying an older number forward. Where a component does not work, the manual says that too — Chapter 17 is a register of known defects, and the ablations in Chapter 13 include the components that turned out to contribute nothing.

1.1 Conventions

- Weights are **negative log probabilities** throughout. Lower is better, and costs add. This is the **tropical semiring** (min, +) [Mohri 1997], which is why decoding is a shortest-path problem.
- R denotes a Roman-script input string, D a Devanagari output string, a an *akshara* (orthographic syllable), s a Roman *chunk*.
- Accuracy figures are **top-1 exact string match** unless stated otherwise.
- AK-Freq, AK-NEF, AK-NEI are the three sources of the Aksharantar test split: frequent native words, and two named-entity sets.
- Code references are given as `path/to/file.rs`.

1.2 Reproducing everything in this manual

`data/` is entirely gitignored (§17) — a fresh clone has the code and no data or model. Two things are reproducible from it, and they are different claims: that the code builds and its unit tests pass (seconds, no network), and that the shipped `data/akshar.model` and every number in §2.3/§12 can be regenerated from public data (hours, real bandwidth). Both are exercised below; CI (`.github/workflows/ci.yml`) runs only the first — it does not fetch data, train, or check that the from-scratch sequence below still works.

```
make release      # build the engine -- no data needed (build.rs, §17)
make test         # unit + integration tests, including the accuracy guard
```

From-scratch model, in order (each step is idempotent; re-run any one after a code change without repeating the ones before it):

```
make data-fetch SET=aksharantar      # ~229 MB, pinned + SHA-256 verified
make data-fetch SET=indicorp-v2-sample # ~300 MB/language, same
make data-prepare                    # -> data/pairs/{train,valid,test}.jsonl
```

```

make lexicon                # -> data/lexicon.bin (§8.6)
make train-full             # ~4h; train-mid (~40min) for a faster,
  ↳ weaker check
make model                  # calibrate blend + fit the size budget
  ↳ -> data/akshar.model

```

train/train-full/train-mid auto-detect the multi-language pairs and lexicon above over the single-language predecessor if present (whichever exists), and write `data/akshar.model` directly; `make model` (calibrate then promote, each independently re-runnable) is what turns that into the artifact this manual reports on – a raw `train-full` output has neither a calibrated per-language blend (§8.6) nor the desktop size budget applied (§10.4), and the numbers in §2.3 are measured after both.

The full sequence above was not run end-to-end while writing this revision (`train-full` alone is ~4h); what was verified directly is that `train`, `calibrate` and `promote` correctly auto-detect and chain (`cargo run --release --bin train -- --smoke`, a ~6-second synthetic-scale run exercising every stage of the same code path, immediately followed by `make calibrate` and `make promote` against its output). The from-scratch wall-clock times above are the targets’ own documented estimates, not independently timed this session.

Measuring it:

```

make eval-langs             # §2.3's table: per-language top-1/in-list/MRR
  ↳ (needs data-prepare)
cargo run --release --bin eval_session -- --lang-aware # §12.10: cold vs.
  ↳ session accuracy
make eval                  # legacy single-language accuracy by split
make eval-full             # legacy: bootstrap CIs and per-query latency
make eval-errors           # legacy: oracle curves, error taxonomy, CER, collision bound

```

Component ablations are switched at runtime; see §13.1.

Chapter 2

What this system is

Akshar is an input method engine that converts Roman-script typing into Devanagari. You type `namaste` and it offers नमस्ते.

It is explicitly **multi-language**, not script-generic with one language’s priors leaking through a “language-agnostic” label. One phonetic engine (akshara units, script-wide emissions) is shared by eight Devanagari languages — Hindi, Marathi, Nepali, Sanskrit, Konkani, Maithili, Bodo, Dogri — and a language tag conditions ranking explicitly (§8.6, LangCond) rather than being absent from the pipeline. Earlier revisions of this manual described the engine as language-agnostic; that was true of the phonetics but concealed a single Nepali-only frequency vocabulary underneath, which capped every other language’s accuracy regardless of the phonetics (§17.1 logs the correction rather than quietly dropping the old claim).

Three other properties define the design:

No neural network at runtime. The model is an EM-trained source-channel model over orthographic syllables, a Kneser-Ney syllable language model, and a linear discriminative reranker. Inference is beam search plus dot products. There is no tensor library and no GPU. Runtime dependencies are the Rust standard library plus maintained crates (`serde`, `unicode-normalization`, `fst`, `smallvec`); developer tooling additionally uses `clap`, `rand` / `rand_chacha` and `unicode-segmentation` (evaluation harness) and `criterion` (benchmarks).

Small enough to ship in a web page. The desktop container is 24.44 MB for all eight languages (one shared automaton, not eight separate models — §8.6); the browser profile is 23.30 MB raw and 13.97 MB Brotli-compressed.

Fast enough to run on every keystroke. A suggestion query costs **1.16 ms** end to end on a desktop CPU with a beam width of 32 (§14) — slower in absolute terms than the single-language, dictionary-free predecessor (0.63–0.82 ms), because every query now also walks a shared dictionary automaton across eight languages’ vocabularies; still three orders of magnitude under any perceptible-lag threshold.

2.1 Contribution and prior art

None of the individual techniques here are new. Source-channel transliteration over EM-aligned units, Kneser-Ney n-gram smoothing, linear discriminative reranking, and WFST/lexicon-constrained decoding are all textbook, and FST-based statistical transliteration for Devanagari specifically has direct prior art (e.g. Malik et al.’s Hindi–Urdu FST cascades; Kunder’s Konkerverter for Konkani). What we believe is a more specific, genuinely assembled contribution, stated with the hedging a paper needs and *not* independently literature-searched beyond the general web checks cited in §20:

1. **One shared multi-language lexicon automaton**, not eight per-language ones. Every word of every language lives in a single minimal FST keyed on Devanagari codepoints (one byte each), with a per-language quantised frequency level packed into a de-duplicated palette (§8.6, `lexicon.rs`). Related-but-different prior art exists for lexical sharing across Indic languages in *machine translation* (e.g. arXiv:2305.03207); we have not found the equivalent for a transliteration *decoding-and-ranking* lexicon. Stated precisely, since we checked rather than assumed it (§8.7): the measured benefit is compactness and one code path, not cross-lingual accuracy transfer to the low-resource languages sharing the automaton with better-resourced ones — swapping Bodo/Dogri onto an isolated, two-language automaton changed their test hit counts by 0 and 1-in-2,000 respectively. We are not claiming a transfer-learning effect that the data does not support.
2. **A session-level, learning-aware evaluation** (`eval_session`, §12.10, added while preparing this document) that samples words by real corpus frequency and replays them through an engine that actually calls `user_confirms`, the same path the shipped front ends use. Every transliteration paper we are aware of, including IndicXlit’s own ([Madhani et al. 2023]), reports a single cold, uniformly-weighted top-1 number. That is the right number for cross-paper comparison (we report it too, unchanged in methodology, immediately below) — but it is not what a returning user experiences, and the gap turns out to be large (§12.10).
3. **What this is not**: a claim of beating neural SOTA on the comparable metric. On cold, uniform top-1 — the only number directly comparable across systems — this system trails IndicXlit’s word-LM-reranked numbers in most of the eight languages (table below) and is only close for Nepali. The defensible claim is architectural: comparable ballpark accuracy for less commonly-served languages, at no neural runtime, ~24 MB, sub-2ms CPU decode, versus an ~11M-parameter transformer whose own shipped/quantised artifact size we have not measured and do not claim to beat.

2.2 What it is not

It does not beat IndicXlit’s word-LM-reranked top-1 in most of the eight languages below — only Nepali is close on the unreranked column. Named-entity accuracy is far behind native-word accuracy in every language. Bodo and Dogri are specifically limited by how little clean training text exists for them (34,480 and 1,276 pairs after cleaning — §12.1); their generation ceiling, not just their ranking, is measurably lower than the other six languages’ (§12.2, the `in@50` column). And the cold, uniform top-1 number this manual leads with — the one comparable to published work — understates what a *returning* user experiences by a wide margin (§12.10).

2.3 Measured performance

On the held-out multi-language test split (34,011 native-word cases + 14,266 named-entity cases across all eight languages, `data/pairs/test.jsonl` built by `make data-prepare` — §12.1), measured on `data/akshar.model`, language-aware (`--lang-aware, make eval-langs`):

Lang	native <i>n</i>	native top-1	in-list@8	entity <i>n</i>	entity top-1	IndicXlit AK-Freq top-1 (plain / +word LM)
Hindi (hin)	8,098	53.42%	81.45%	2,014	43.59%	58.6 / 67.9
Marathi (mar)	10,112	66.78%	83.79%	2,078	37.15%	74.7 / 85.5
Nepali (nep)	2,108	78.32%	92.55%	1,993	34.97%	80.2 / 86.6

Lang	native n	native top-1	in-list@8	entity n	entity top-1	IndicXlit AK-Freq top-1 (plain / +word LM)
Sanskrit (san)	2,927	75.30%	92.07%	2,375	18.48%	81.6 / 90.1
Konkani (kok)	3,051	56.24%	79.15%	1,991	30.94%	65.4 / 76.3
Maithili (mai)	3,471	69.12%	90.98%	1,978	38.78%	78.7 / 87.6
Bodo (brx)	2,244	41.67%	56.60%	1,837	18.40%	74.8 / 78.4
Dogri (doi)	2,000	32.80%	54.20%	—	—	—
macro		59.21%	78.85%		31.76%	
pooled	34,011	60.69%	81.27%	14,266	31.59%	

Lenient scoring (nukta-optional, chandrabindu/anusvara, nasal-cluster/anusvara equivalence — §12.2) adds +2.27pp pooled native top-1 (62.96%) and +1.14pp pooled in-list (82.41%).

For reference, **IndicXlit** (AI4Bharat, an ~11M-parameter transformer trained on the full 26M-pair Aksharantar corpus) reports the AK-Freq column above without LM reranking, and with its word-unigram rerank alongside it ([Madhani et al. 2023], Table 13). Neither IndicXlit figure is re-measured here; both are the paper’s own numbers, reproduced as a reference column, not a head-to-head run under identical conditions.

This is the cold, uniformly-weighted number: every distinct word counted once, no memory between queries. §12.10 measures the same model under realistic, frequency-weighted, learning-enabled use and finds pooled top-1 rises to **97.28%** — the gap is the subject of that section, not a contradiction of the table above.

Resource	Desktop	Browser
Container size	24.44 MB	23.30 MB
Brotli-compressed	not applicable (native binary)	13.97 MB
Query latency	1.16 ms end to end ($k = 8$, beam 32 — §14)	not re-measured on this artifact
Cold start	not re-measured on this artifact	not re-measured

Chapter 3

Quick start

3.1 Build and install (Linux / IBus)

```
make release      # builds libakshar_ime.so and the IBus C engine
sudo make install # installs to /usr/lib/ibus/engines and /usr/share
make restart-ibus # NOT as root
```

Then add “Akshar Devanagari” as an input source in your desktop settings.

3.2 Use from Rust

```
use akshar_ime::ImeEngine;

let mut engine = ImeEngine::new(); // loads data/akshar.model
let suggestions = engine.get_suggestions("namaste", 5);
for (devanagari, score) in &suggestions {
    println!("{devanagari} {score}");
}

// Teach the engine what the user actually picked.
engine.user_confirms("namaste", "नामस्ते");
```

3.3 Use from the browser

```
make wasm          # builds packages/engine-wasm/pkg/
```

```
import { AksharIME } from 'packages/engine-js/akshar-ime.js';
await AksharIME.init({ modelUrl: './models/akshar_wasm.model' });
AksharIME.attach(document.querySelector('input'));
```

The runnable playground (editor + deploy) ships from a separate repository (self-contained: engine, wrapper, model bundled, no CDN) — see docs/plans/playground-extension.md. This repo builds the engine it embeds.

Chapter 4

Problem formulation

4.1 The task

Given a Roman string R typed by a user, produce a ranked list of Devanagari strings D that the user plausibly meant. This is *transliteration*, not translation: the output should be the same word in a different script.

The difficulty is that Roman input for Devanagari is **not a code**. It is a lossy, inconsistent, user-invented approximation:

- Vowel length is routinely dropped. *sathi* and *saathee* are both साथी.
- Retroflex/dental distinctions collapse. *t* may be त or ट.
- Aspiration is inconsistent. *kh* may be ख, but *k* sometimes is too.
- The inherent schwa is written or omitted at the user's discretion: *kamal*, *kama la*, and *km l* all target कमल.
- Conjuncts have no standard Roman form.

So the mapping is many-to-many in both directions, and the model must learn the conventions from data rather than encode them by hand.

4.2 The source-channel decomposition

Following Li, Zhang & Su (ACL 2004), *A Joint Source-Channel Model for Machine Transliteration*, we model the user as a noisy channel: they have a Devanagari word D in mind and emit a Roman rendering R of it. Then

$$\hat{D} = \arg \max_D P(D | R) = \arg \max_D \underbrace{P(R | D)}_{\text{channel}} \cdot \underbrace{P(D)}_{\text{source}}.$$

The two factors are learned separately:

- $P(R | D)$ — the **transliteration model**, learned by EM over an unaligned parallel lexicon (Chapter 5).
- $P(D)$ — a **language model** over aksharas, smoothed with modified Kneser-Ney [Kneser & Ney 1995; Chen & Goodman 1999] (Chapter 6).

Working in negative logs, the decoder minimises

$$\text{cost}(D) = \underbrace{-\log P(R | D)}_{\text{emit}} + \lambda \cdot \underbrace{-\log P(D)}_{\text{lm}},$$

with λ (`DecoderConfig::lm_weight`, default 1.0) a tunable balance. The two terms are accumulated **separately** all the way through decoding, because the discriminative reranker (Chapter 7) consumes them as independent features.

4.3 The unit of modelling: the akshara

Neither characters nor whole words are the right granularity.

Characters are wrong because Devanagari is an *abugida*: a consonant carries an inherent vowel, and a matra (vowel sign) modifies the preceding consonant rather than standing alone. The codepoint sequence क + ि is one pronounceable unit, कि, and splitting it produces meaningless states.

Whole words are wrong because the vocabulary is open — the language is agglutinative, and compounds and case-marked forms are productive.

So the unit is the **akshara**, the orthographic syllable of Brahmic scripts. The script class is Daniels’ *abugida* [Daniels 1990]: a consonant carries an inherent vowel that a diacritic overrides.

akshara := (consonant halanta)* consonant? (matra | independent-vowel) (anusvara | visarga | chandrabindu | nukta)*

Implemented in `src/core/akshara.rs`. Examples:

Word	Aksharas
नमस्ते	न म स्ते
क्ष्त्र	क्ष्त्र (one conjunct)
अग	अ ग
काठमाडौ	का ठ मा डौ

A halanta (virama, ँ) glues the following consonant into the current akshara, which is what makes conjuncts single units. Zero-width joiners are preserved when attached to viramas, so eyelash-ra ऌ keeps its visual form.

The trained model has **16,556 aksharas** and **101,010 Roman chunks** before pruning; after pruning to those reachable from the corpus vocabulary, 5,938 emission rows remain.

4.4 Alignment is latent

The training data is a list of (R, D) pairs with **no alignment** between Roman substrings and aksharas. `kathmandu / काठमाडौ` does not say that `kath` corresponds to `का ठ`.

So the model treats the segmentation as a latent variable and sums over all of them:

$$P(R | D) = \sum_{\text{segmentations } s_1 \dots s_n \text{ of } R} \prod_{j=1}^n P(s_j | a_j),$$

where $a_1 \dots a_n$ are the aksharas of D and each s_j is a contiguous (possibly empty) chunk of Roman characters, of length at most `MAX_CHUNK = 5`. This sum is computed exactly by dynamic programming, and EM maximises it (Chapter 5).

Chapter 5

The transliteration model and EM training

Implemented in `src/core/em_trainer.rs`, `src/core/alignment.rs`, and `src/core/translit_model.rs`.

5.1 Parameters

The model is a single table:

$$\theta_{a,s} = P(s \mid a), \quad \sum_s \theta_{a,s} = 1 \quad \forall a,$$

the probability that akshara a is written as Roman chunk s . Chunks are lowercase ASCII. The training lattice permits lengths $0 \dots 5$ (`em_trainer.rs:9`); the shipped vocabulary holds lengths $1 \dots 5$ (`translit_model.rs:45`). The empty alignment is how training explains aksharas with no Roman counterpart; every chunk consumed at decode time has length ≥ 1 .

Chunks are packed into a u32 for allocation-free lookup: 5 bits per character (26 letters plus an escape) in bits 0–24, and a 4-bit length in bits 26–29 (`translit_model.rs::pack_chunk_bytes`).

5.2 Initialisation

EM is sensitive to its starting point, so emissions are seeded from a deterministic codepoint aligner (`alignment.rs::align_emissive`) rather than from a uniform distribution. For each pair, the aligner walks both strings and assigns each akshara the Roman characters that plausibly correspond to it.

Bare consonants additionally seed two variants:

- the chunk plus a (the inherent schwa written out: $\text{च} \rightarrow \text{cha}$), and
- the chunk minus a trailing a (the schwa dropped),

so that EM has both conventions available from the first iteration rather than having to discover one from a distribution that assigns it zero mass.

Seed counts are normalised and floored at 10^{-4} so no observed chunk starts with probability zero.

5.3 The E-step: scaled forward-backward

For a pair with Roman length m and n aksharas, define

$$\alpha_j(i) = P(\text{first } i \text{ Roman chars generated by first } j \text{ aksharas}),$$

$$\beta_j(i) = P(\text{Roman chars } i+1 \dots m \text{ generated by aksharas } j+1 \dots n).$$

with the recurrences

$$\alpha_j(i) = \sum_{l=0}^{\min(5, i)} \alpha_{j-1}(i-l) \theta_{a_j, R[i-l..i]}, \quad \beta_j(i) = \sum_{l=0}^{\min(5, m-i)} \theta_{a_{j+1}, R[i..i+l]} \beta_{j+1}(i+l),$$

$\alpha_0(0) = 1, \beta_n(m) = 1$. The total likelihood is $Z = \alpha_n(m)$, and the expected count of (a_j, s) at position i is

$$\gamma_j(i, l) = \frac{\alpha_{j-1}(i-l) \theta_{a_j, R[i-l..i]} \beta_j(i)}{Z}.$$

5.3.1 Why the passes are scaled

Computed naively, Z underflows. A 12-akshara word whose emissions average 0.05 gives $Z \approx 2.4 \times 10^{-16}$. An earlier implementation guarded this with `if z < 1e-12 { continue; }` — a threshold 296 orders of magnitude above the f64 subnormal limit — which silently **discarded every long or flat-emission word from training**, biasing EM toward short easy words.

The fix is the scaling of [Rabiner 1989, §V.A], developed there for HMM forward-backward [Baum et al. 1970]. Each forward column is divided by its own maximum c_j , and **the same factors** are divided out of the backward pass, so that with

$$F_j = \alpha_j / \prod_{t \leq j} c_t, \quad B_j = \beta_j / \prod_{t > j} c_t,$$

the scale products telescope and the posterior needs only a single per-column correction:

$$\gamma_j(i, l) = \frac{F_{j-1}(i-l) \theta B_j(i)}{c_j \cdot F_n(m)}.$$

Derivation: $\alpha_{j-1} = F_{j-1} S_{j-1}$ and $\beta_j = B_j S_n / S_j$ with $S_j = \prod_{t \leq j} c_t$, and $Z = F_n(m) S_n$. Substituting, the S factors cancel to $1/c_j$ because $S_j = S_{j-1} c_j$. $\square$

This is verified by test, not by inspection: `scaled_forward_backward_matches_unscaled` compares against an unscaled reference implementation, `posteriors_sum_to_weight_per_akshara` checks that each akshara’s posterior mass equals its observation weight, and `long_low_probability_word_is_not_dropped` pins the 12-akshara case the old floor discarded.

The E-step is embarrassingly parallel over pairs and runs on scoped threads.

5.4 The M-step

Counts are normalised with Dirichlet smoothing toward the global chunk unigram $u(s)$, so that a chunk seen with one akshara does not receive zero probability under another:

$$\theta_{a,s} \leftarrow \frac{\text{count}(a, s) + \alpha u(s)}{\sum_{s'} \text{count}(a, s') + \alpha}, \quad \alpha = 0.05.$$

Twelve iterations are run by default. EM over all 3,588,793 pairs takes about 170 s on a desktop CPU.

5.5 What the model looks like when trained

Emissions are stored as `Vec<Vec<(chunk_id, -log P)>>`, one row per akshara, each row **sorted ascending by chunk id**. That ordering is not incidental: the container delta-encodes the ids (§8.2) and the runtime binary-searches them (§10.2), and both would be incorrect for any other order.

Chapter 6

The language model

Implemented in `em_trainer.rs::build_kn_lm` and consumed by `translit_model.rs`. This is an akshara n -gram model, and it is **the largest single contributor to accuracy** in the system: removing the trigram order costs 3.89pp of native top-1 (§9.1).

6.1 Why Kneser-Ney

Maximum-likelihood n -gram estimates assign zero to unseen contexts, and add- α smoothing is badly wrong on large alphabets — with 16,556 aksharas, a single observation under add-1 would imply $P \approx 0.67$.

Kneser-Ney addresses a subtler problem. The right lower-order estimate is not “how often did this akshara occur” but “**in how many distinct contexts** did it occur”. An akshara that appears constantly but always after the same predecessor (the *Kong* in *Hong Kong*) is a poor bet in a novel context. The continuation probability captures exactly that:

$$P_{\text{cont}}(b) = \frac{|\{a : c(a, b) > 0\}| + 0.5}{|\{(a, b) : c(a, b) > 0\}| + 0.5N},$$

with the $+0.5$ floor keeping the log finite for aksharas that only ever appear word-initially. N is the akshara vocabulary size.

6.2 Modified Kneser-Ney discounts

A single absolute discount δ [Ney, Essen & Kneser 1994] over-discounts frequent n -grams and under-discounts singletons. Chen & Goodman (1999) use three discounts, chosen by the count of the n -gram, estimated from the counts-of-counts $n_1 \dots n_4$:

$$Y = \frac{n_1}{n_1 + 2n_2}, \quad D_1 = 1 - 2Y \frac{n_2}{n_1}, \quad D_2 = 2 - 3Y \frac{n_3}{n_2}, \quad D_3 = 3 - 4Y \frac{n_4}{n_3},$$

subject to $0 \leq D_i \leq i$. That constraint matters. An earlier implementation clamped all three to $\leq 0.9/0.9/0.95$; the values this corpus actually produces are

order	D_1	D_2	D_3
bigram	0.6049	1.0362	1.4443
trigram	0.6696	1.0949	1.4330

so D_2 was being cut by 13% and D_3 by 34%, both pinned at the ceiling. The effect was to collapse modified Kneser-Ney back into single-discount absolute discounting at $d \approx 0.9$ — *worse* than the fixed 0.75 it replaced. The bounds are now correct. See §13.6 for what fixing them was worth.

If n_1, n_2 or n_3 is zero the estimator is degenerate and the model falls back to fixed (0.5, 0.75, 0.95).

6.3 The interpolated model

For a bigram context a with total count $c(a) = \sum_b c(a, b)$:

$$P(b | a) = \frac{\max(c(a, b) - D_{c(a, b)}, 0)}{c(a)} + \lambda(a) P_{\text{cont}}(b), \quad \lambda(a) = \frac{\sum_b D_{c(a, b)}}{c(a)}.$$

$\lambda(a)$ is exactly the mass removed by discounting, so the distribution sums to one. The trigram has the same form with context (a, b) and $c(a, b)$ as the denominator, backing off to the bigram.

A **word-start prior** is estimated separately from word-initial counts:

$$P(a | \#) = \frac{c_{\#}(a) + 0.5}{W + 0.5N}, \quad W = \text{corpus word count}.$$

6.4 Storage and lookup

All quantities are stored as $-\log$ weights in f32. Per context the model stores the seen successors and one backoff weight $-\log \lambda$; an unseen successor costs $-\log \lambda + (-\log P_{\text{lower}})$, which reconstructs the interpolated value exactly. `trigram_weight` falls back to `bigram_weight`, which falls back to `unigram_kn`.

Successor rows are sorted ascending by id and looked up by **binary search**. This is not a micro-optimisation: the beam performs roughly 50,000 LM lookups per query, and when these rows were scanned linearly they accounted for most of the engine’s runtime (§10.2).

6.5 Two historical deviations from textbook KN (resolved in v1.2.0)

Both were recorded here and resolved in v1.2.0 (§11, §12):

1. **Trigram backoff to continuation estimate:** An earlier version interpolated against the highest-order bigram $P(c | b)$ rather than the continuation estimate $P_{\text{cont}}(c | b) = N_{1+}(\bullet, b, c) / \sum_{c'} N_{1+}(\bullet, b, c')$. In v1.2.0, `Trainer` builds `bigram_cont_right` during pair ingestion and uses the correct textbook continuation count in `build_kn_lm()`.
2. **End-of-word boundary symbol `</w>`:** The model now interns a synthetic `</w>` token appended to each word’s akshara sequence during LM counting (omitted from emission training). Trigram mass now properly closes word-finally, allowing the model to penalise implausible word endings.

Chapter 7

Decoding

Implemented in `src/core/decoder.rs`.

7.1 The lattice

Decoding is a shortest-path search over a lattice built on the Roman string. Nodes are character positions $0 \dots m$. An edge from position p to $p + l$ is labelled with an akshara a that can emit the chunk $R[p..p+l]$, and carries weight $-\log P(R[p..p+l] \mid a)$.

Edges come from a **reverse index** built once at load time: chunk $\rightarrow$ list of $(akshara, weight)$, sorted by weight and truncated. Two caps keep the lattice tractable:

- `max_emission_weight = 8.0` — emissions worse than e^{-8} are alignment noise and are dropped.
- `max_aksharas_per_chunk = 16` — at most 16 aksharas compete for any chunk.

A path from 0 to m spells one candidate Devanagari string; its cost is the sum of edge weights plus the LM cost of its akshara sequence.

7.2 Beam search

The search is a **beam search** [Lowerre 1976], step-synchronous over aksharas. Each step expands every beam state across every edge available at its position, scores the extension, and keeps the best `beam_width` hypotheses (default 64, `AKSHAR_BEAM`).

A state carries its position, its previous two aksharas (the trigram context), the running emit and lm costs **separately**, a path hash, and an index into a persistent path arena. Extending a path is $O(1)$: one arena cell holding `(parent, akshara)`. Reconstruction walks parents backwards at the end.

Three properties of this loop are deliberate and worth stating, because each looks like an oversight:

Distinct paths are not merged by state. A conventional Viterbi beam would collapse hypotheses sharing `(pos, prev2, prev)` and keep the best. This decoder extracts *k-best paths*, not the 1-best path per state, so merging would destroy the diversity the reranker needs. An earlier version keyed a merge map on `(pos, prev2, prev, path_hash)`, which could only ever merge on a hash collision — it built and tore down a hash map every step to do nothing, and was removed.

Pruning uses `select_nth_unstable_by`, not a sort. Keeping the best 64 of ~5,000 hypotheses does not require ordering them; the beam’s internal order is never read. This is $O(n)$ instead of $O(n \log n)$.

Arena cells are materialised only for survivors. Hypotheses are generated into a lightweight `Cand` struct holding the parent index and its own akshara; only after pruning are the survivors written into the arena.

This cuts arena writes from $O(\text{beam} \times \text{edges})$ to $O(\text{beam})$ per step.

Completed paths (those reaching position m) are collected into a map keyed by the output string, keeping the best cost per string, and sorted once at the end.

7.3 Two passes: free and dictionary-constrained

`decode_union` runs the beam **twice** and merges:

1. **Free decode** (`decode_detailed`) — the full lattice. Can produce any akshara sequence, including words that do not exist. This is what handles novel compounds and out-of-vocabulary names.
2. **Dictionary-constrained decode** (`decode_in_words_detailed`) — the lattice intersected with a Dictionary: every edge must extend a valid word prefix, and only dictionary terminals are returned.

Dictionary is a trait (`root`, `step`, `is_word`), not a concrete structure, because container v8 (§10.2) replaced the single-language `WordTrie` with `LexiconDict`, a thin walk over the shared multi-language automaton (`lexicon.rs`, described fully in §8.6 since it is also the ranking frequency source). Older model files still carry a `WordTrie`, which implements the same trait, so this pass’s logic did not change when the lexicon shipped — only which structure it walks did. A model loads exactly one of the two, never both.

The second pass is cheaper than the free pass because most akshara sequences are not word prefixes, so its beam collapses almost immediately: measured 0.375 ms against 0.576 ms on the eight-language `LexiconDict` (§14; the original single-language `WordTrie` measured 0.12 ms against 0.55 ms, a narrower gap because that dictionary held one tenth as many words). It is also strictly a recall addition: it surfaces real words the free beam ranked outside its top 50.

Its contribution is +0.66pp top-1 and +1.14pp top-5 on the original single-language harness (§9.1). Running *only* the dictionary pass would be faster still, but costs double-digit points of native top-1 — the dictionary does not cover every valid novel form, and never will — so both passes stay.

7.4 What the decoder returns

A `DecodedCandidate` per output string, carrying `emit`, `lm` and `akshara_count` **separately** rather than as one summed score, because the reranker uses them as independent features.

Chapter 8

Discriminative reranking

Implemented in `src/core/reranker.rs`, weights in `reranker_weights.rs`.

The decoder’s generative score is not the last word. It knows nothing about how common a word is, what shape words of the language have, or how its morphology works. The reranker re-scores the decoder’s top candidates using features the generative model cannot express.

8.1 The baseline heuristic

Before any learned model, candidates are scored by a three-parameter heuristic:

$$h(D) = \text{emit} + 0.85 \cdot \text{lm} - 0.75 \cdot \log(1 + f(D)),$$

where $f(D)$ is the corpus frequency of D . This alone achieves **81.02%** native top-1 — it is a strong baseline, and the learned model must be measured against it, not against nothing.

8.2 Dense features

29 features per candidate (`extract_dense_features`):

Index	Feature
0–1	emit, lm from the decoder
2	akshara count
3	decoder rank
4–5	heuristic score, heuristic rank
6–8	$\log(1 + f)$, frequency-rank percentile, in-vocabulary indicator
9	output length in characters
10	total matra count
11–20	per-matra counts (10 matras)
21–23	nasals, visarga, halants
24–26	vowel-initial, ends-in-matra, ends-in-nasal/visarga
27	Roman input length
28	morphology-aware effective log frequency

Feature 28 deserves a note. The language is agglutinative, so an inflected form may be absent from the corpus while its stem is frequent. `morph_effective_log_freq` strips one of 34 known suffixes (को, हरू, लाई, एको, ...) and, if the stem has frequency ≥ 5 , returns $\log f(\text{stem}) + 3.5$. This gives unseen but well-formed inflections a frequency prior instead of zero.

Features are standardised before scoring:

$$\text{score}_{\text{dense}}(D) = \sum_{k=0}^{28} w_k \cdot \frac{\phi_k(D) - \mu_k}{\sigma_k}.$$

μ and σ **travel with the model**, in the v5 container. They used to be compiled-in constants that no training stage refreshed — and since retraining the EM/LM shifts the `emit` and `lm` distributions (a 100k retrain moved `lm` from $\mu = 21.83, \sigma = 5.74$ to $24.64, 7.03$), the weights were being applied to differently-scaled inputs with nothing to detect it. Containers older than v5 fall back to the constants.

8.3 Sparse lexicalized features

Seven templates are hashed into a 2^{20} -slot table of 18 weights — the **hashing trick** [Weinberger et al. 2009], which trades hash collisions for a fixed memory budget over an unbounded feature space:

1. length-delta bucket (aksharas minus Roman characters)
2. final akshara $\times$ final Roman character
3. first akshara $\times$ first Roman character
4. matra $\times$ preceding consonant
5. morphological suffix $\times$ Roman tail character
6. final matra $\times$ final Roman character
7. penultimate consonant $\times$ final matra

Weights are quantised to 18 with the scale set from the 99.9th percentile of non-zero magnitudes and explicit clipping — not from the single largest weight, which previously let one outlier compress every other weight into a handful of levels.

8.4 The blend

$$\text{score}(D) = (1 - \gamma) \cdot (-z(h(D))) + \gamma \cdot z(\text{score}_{\text{dense+sparse}}(D)), \quad \gamma = 0.3,$$

where $z(\cdot)$ standardises within the candidate list. γ was tuned, and is worth understanding honestly:

γ	meaning	AK-Freq top-1
0.0	heuristic only, learned model unused	81.02%
0.3	shipped	81.83%
0.6		81.31%
1.0	learned model only	76.47%

The learned model used alone is 5.36pp worse than the three-parameter heuristic. $\gamma = 0.3$ is not a cautious discount of a good model; it is the blend point at which a weak model stops doing damage.

The cause is identified in §18.2: `w_DENSE` has never been retrained by this pipeline. It is a frozen constant, and the generator named in its header does not exist in the repository. Training more sparse capacity on top of misfitted dense weights was tested at 5x the data and did not help.

8.5 The cascade

Full feature extraction — akshara segmentation, matra scans, sparse hashing — is far more expensive than the heuristic, and candidates the heuristic already ranks far down never reach the top of the final list. Only the top `AKSHAR_RERANK_DEPTH` (default 24) by heuristic rank are fully scored; the rest keep heuristic order below the scored block.

Measured on AK-Freq: depth 24 gives 81.83/92.22, depth 50 (no cascade) gives 81.69/92.22, depth 12 gives 81.78/92.41, depth 6 gives 81.31/91.98. The effect is within noise down to depth 12.

One subtlety is documented in the source: the synthetic scores given to unscored candidates participate in the γ z-blend, so the blend weight moves slightly with the scored/unscored ratio. Ordering among scored candidates is unaffected (z-scoring is affine).

8.6 Language-conditioned ranking and the shared lexicon

§8.1 and §8.2’s frequency features (`f(D)`, feature 6–8) came from a single `HashMap<String, u32>` built from one corpus. That is where the old “language-agnostic” framing (§2) actually broke: Nepali’s vocabulary is not Hindi’s, so ranking every language against one frequency table capped every language but the one the table was built from, regardless of how good the phonetics were underneath. Fixing this needed two changes, not one — a place to *put* eight languages’ frequencies, and a way for the ranker to *use* the right one.

The lexicon (`src/core/lexicon.rs`). Every word of every language lives in one minimal acyclic FST (the `fst` crate), keyed one byte per Devanagari character (codepoint - `0x880`; every character in scope is `U+0900–U+097F`, so this is exact and three times denser than UTF-8). The value is a palette index into `Vec<[u8; 8]>` — eight quantised frequency levels, one per language, de-duplicated so the ~65k distinct level-vectors across 1.2M+ words are stored once each. Quantisation is log-scale, normalised to a common per-100M-token basis so languages of very different corpus size are comparable:

$$\text{level} = \text{round}(8 \cdot \log_2(1 + c \cdot 10^8 / N)), \quad c = \text{count}, N = \text{corpus tokens},$$

clamped to `[1, 255]`, with 0 reserved for “this language does not use this word.” Eighth-octave steps ($2^{1/8} \approx 9\%$) are finer than corpus sampling noise, so nothing is lost to the quantisation. Built by `build_lexicon` from an IndicCorp v2 sample per language (§12.1), capped to the 300,000 most frequent words per language above a floor of 2 occurrences — both to bound automaton size and because words rarer than that are noise relative to what an 86M-sample-scale corpus can resolve. Same automaton is consulted twice: `Lexicon::level/count` for the frequency features above, and, via the `Dictionary` impl in `decoder.rs` (§7.3), as the dictionary the second decode pass is constrained to. One artefact does both jobs a per-language `WordTrie` and `HashMap` used to do separately, which is most of why the container grew by less than the eight-fold word count might suggest (§10.2).

WordCounts, a trait with `count/rank_pct`, lets the reranker’s feature extraction and the decoder’s frequency heuristic (§8.1) run unchanged over either a `Lexicon` (`LexiconCounts`, v8 containers) or the legacy single-corpus `HashMap` (`VocabCounts`, older containers) — the same abstraction-over-two-implementations pattern as `Dictionary`.

Feature augmentation [Daumé III 2007] conditions ranking on the query’s declared language without training eight separate rerankers. Each sparse template (§8.3) is hashed twice per candidate: once into a *shared* slot every language’s training data updates, and once into a *language-tagged* copy only that language’s data touches. A query ranked with a known language reads both tables (language-specific signal where there is enough data, shared signal as a backoff everywhere else); a language-blind query (`set_language(None)`) reads only the shared table. Dense features get the same treatment at a coarser grain: a 29×8 `dense_lang_weights` matrix of per-language offsets added to the shared dense weights.

Training applies **language dropout** (each example’s language tag is withheld from the shared/tagged split some fraction of the time) so the shared table cannot collapse into memorising whichever language happens to dominate the training mix — the same failure mode §11’s trainer used to have systemically, in three independent forms (§17.1, D19–D21).

γ (the heuristic/learned blend, §8.4) is likewise calibrated per language on the validation split (`calibrate_blend`, `gamma_lang`) rather than shared, since languages with less training data trust the learned model less. A query with no language falls back to `gamma_auto`, the old language-blind calibration.

Measured, language-aware vs. the phonetic engine with all of this disabled (`AKSHAR_NO_RERANK=1`, so ranking is decoder score alone, no lexicon, no reranker) — see §12.10 for why this comparison is the direct answer to “is a purely computational, dictionary-free engine possible here”: macro top-1 on native-word validation cases rises from **~50.3%** to **~64.4%** across the eight languages. The gap is not an engineering shortcoming of the phonetic model; §12.9’s collision-bound analysis already established that Roman spellings are genuinely ambiguous between multiple valid Devanagari renderings, and only usage frequency disambiguates them.

8.7 Does the shared automaton transfer, or just compact? (ablation)

One automaton for eight languages could help a low-resource language two structurally different ways, and it is worth being precise about which one this system actually gets, because only one of them is true.

Ranking cannot leak across languages by construction. A language-aware query resolves its frequency through `lexicon.lang_index(code)` into that language’s own byte of the level vector (§8.6); a word absent from Bodo’s counts reads level 0 regardless of how common it is in Hindi. This is not an empirical finding, it follows from `pick`’s code (`lexicon.rs`) reading one index.

Generation can, in principle: `LexiconDict::is_word(decoder.rs)` checks only whether the FST node is final — true if *any* language attested the word, not the query’s language specifically. A word too rare to independently clear Bodo’s own frequency floor could still be dictionary-reachable for a Bodo query if Hindi or Marathi attested it. This is a real code path, not a hypothesis, and it is the one place cross-lingual transfer could actually happen in this architecture.

Measured, whether it does: built an isolated lexicon from only Bodo and Dogri’s IndicCorp text (`build_lexicon --raw <bd+dg-only dir>`, no other language present at all: 70,122 words, 0.50 MB against the full lexicon’s share of the two), swapped it into a copy of `data/akshar.model` in place of the shared one (`examples/swap_lexicon.rs`, EM/LM/reranker weights byte-identical), and ran `eval_langs --langs brx,doi --lang-aware --ceiling 50` on both:

	Bodo top-1	Bodo in-list@8	Dogri top-1	Dogri in-list@8
shared (8 languages)	935/2244	1270/2244	656/2000	1084/2000
isolated (brx+doi only)	935/2244	1270/2244	657/2000	1088/2000
difference	+0	+0	+1	+4

Bodo’s hit counts are **exactly identical**, case for case; Dogri’s differ by 1 and 4 cases out of 2,000 (well inside one standard error, $\approx \sqrt{0.5 \cdot 0.5/2000} \approx 1.1\text{pp} \approx 22$ cases). The generation-side transfer path is real code, but it fires on essentially nothing here.

Why, given the mechanism genuinely exists: `build_lexicon`’s frequency floor is `count >= 2` (§12.1) — a very low bar against Bodo’s 2.3M-token sample. For the shared automaton to rescue a word, it must be

genuine, correct Bodo vocabulary that occurred *exactly once* in Bodo’s own sample (or not at all) while also clearing another language’s floor — a narrow intersection that this test suggests is close to empty in practice. Bodo being Sino-Tibetan (Bodo-Garo) rather than Indo-Aryan like the other seven plausibly narrows it further: less shared tatsam vocabulary to inherit.

So, precisely: the shared automaton’s measured benefit is compactness (one 24.44 MB container instead of an estimated eight separate ones, §10.3) and one code path (§7.3, §8.6), not cross-lingual transfer for the low-resource languages that would most want it. §2.1’s contribution claim is deliberately scoped to the former and does not claim the latter — this ablation is why.

Chapter 9

The engine: candidate fusion and learning

Implemented in `src/core/engine.rs`.

9.1 Sources of evidence

The engine merges candidates from several sources into one ranked list, taking the **maximum** evidence per output string:

Source	Score	Notes
Decoder + reranker	$800,000 / (1 + \text{cost})$	cost is the reranker margin below the best
User-confirmed trie	$900,000 + \text{freq}$	prefix match on learned words
User fuzzy (SymSpell)	$50,000 - 12,000 \cdot d$	distance-verified, learned words only
Context re-rank	multiplicative	user bigrams

This score space is the weakest part of the architecture, and this manual does not pretend otherwise.

The bands are hand-tuned u64 constants compared against a hyperbolically squashed reranker score, and two of the three serious defects found in this system came from that design:

- A corpus-wide fuzzy source scored at 850,000 — above the decoder band — so any fuzzy hit displaced the reranker’s top-1. It cost **30.8pp** of native top-1 and, when tested fairly, contributed no recall at any band. It was removed.
- The user fuzzy band, at 38,000 for a distance-1 match, sits *below* the decoder’s eighth-ranked candidate (~250,000). A word the user has confirmed cannot be recovered from a typo. The path is structurally unreachable (defect D18, §11).

The correct fix for both is not a better constant but a single log-linear score in which every source contributes a *feature* rather than occupying a band. That work is §12.

9.2 Digits and punctuation

Digits map to Devanagari numerals (123 → १२३) and a trailing . offers purnabiram (namaste . → नमस्ते ।). These are handled directly, before the statistical path.

9.3 Adaptive learning

`user_confirms(roman, devanagari)` records the user's choice into:

- the **trie** [Fredkin 1960] (`src/core/trie.rs`) — prefix lookup with frequency;
- the **SymSpell index** (`src/fuzzy/sympell.rs`) — delete-variant map for typo tolerance;
- the **context model** (`src/core/context.rs`) — user bigrams for re-ranking by preceding word.

Learned state is separate from the model and persists via `src/persistence.rs` (desktop) or `export_state/import_state` (browser).

9.3.1 How fuzzy matching actually behaves

Two mechanisms remain, and they are different in kind:

1. **Query-variant rewriting** (`src/core/normalizer.rs`) — a Dijkstra search over a small rewrite transducer ($ee \rightarrow ii$, $oo \rightarrow uu$, $w \rightarrow v$, separator removal), each rule carrying a cost. Cold-start: needs no learning. Measured contribution on the benchmark: **0.00pp**, because only the first variant is ever decoded and the EM emissions already absorb these alternations.
2. **User SymSpell** — symmetric-delete matching [Garbe] over confirmed words. Every hit is verified with bounded Levenshtein distance before scoring, because a delete-set intersection is a *necessary* condition only: lookup returns pairs at true distances beyond the setting (defect D2 in the archive), and ours is `MAX_EDIT_DISTANCE = 2` (`src/core/engine.rs:34`).

Behaviour is pinned by `tests/fuzzy_behavior.rs`.

Chapter 10

The model container

Implemented in `src/core/unified.rs` and `src/core/codec.rs`.

10.1 One file

The engine loads exactly one artefact, `data/akshar.model`. It holds:

- the transliteration model (aksharas, chunks, emissions),
- the Kneser-Ney LM (unigram continuation, bigrams, trigrams, backoffs, word-start prior),
- the corpus vocabulary with frequencies,
- the sparse reranker table and its dequantisation scale,
- the dense-feature normalisation statistics (μ, σ).

10.2 Versions

Version	Change
v1	initial layout
v2	adds <code>sparse_scale</code>
v3	compact codec: CSR adjacency, delta varints, 8-bit codebooks
v4	removes the word-bigram table (19.5 MB for +0.16pp)
v5	carries the reranker's dense normalisation statistics
v6	adds jointly-trained <code>dense_weights</code> (§17.1, <code>W_DENSE</code> trap)
v7	adds <code>langs</code> , <code>dense_lang_weights</code> , <code>gamma_auto</code> , <code>gamma_lang</code> (§8.6)
v8	adds <code>lexicon: Option<LexiconData></code> (§8.6), which supersedes <code>vocab_freq</code> when present

`save` writes v8; v1–v7 still load, each falling back the way its own row says (a v8 reader without a `lexicon` field present, i.e. any file older than v8, reconstructs the legacy `wordTrie/HashMap` path instead — §7.3's `Dictionary` trait is what lets both paths share one decoder). `bincode` is positional and not self-describing, so each version has its own struct and the reader dispatches on a version peeked from the header — appending a field to an existing struct would silently corrupt every file already written with it.

10.3 Compact encoding

Measured against the single-language v5 container: naive bincode of those structures was 66.59 MB, and four techniques below brought it to 11.37 MB with no accuracy change. The same techniques apply unchanged to the eight-language v8 container (24.44 MB, §2), which is smaller than eight separate 11.37 MB containers would be because §8.6’s automaton stores every language’s overlapping vocabulary once, not eight times; it is larger than the original 11.37 MB because it holds roughly six times the distinct words and a trigram LM trained on the union of eight languages’ text.

- **CSR adjacency with delta varints.** Successor ids within a row are ascending, so only the gaps are stored, as variable-length integers. (This is also why the rows must stay sorted — see §5.5 and §10.2.)
- **8-bit weight codebooks.** Each numeric section gets a 256-entry codebook of f32 values; entries store an index. Weights cluster tightly, so the quantisation error is far below the model’s discrimination threshold.
- **Front-coded vocabulary** (a standard dictionary compression, as in [Witten, Moffat & Bell 1999]). Words are stored as akshara-id sequences against a shared prefix.
- **Packed chunk strings.**

Section sizes can be inspected at any time:

```
cargo run --release --bin probe_model -- data/akshar.model --inspect
```

10.4 Browser profile

`make web-model` produces `data/akshar_wasm.model` by relative-entropy pruning of the trigram LM [Stolcke 1998] (`prune_lm`, §7.2’s `select_nth_unstable_by` discipline does not apply here — this is a one-time offline pass, not a per-query one). `TRIGRAM_THRESHOLD` (default `3e-2`) trades size against accuracy along a measured curve.

On the single-language v5 container this kept 47% of trigram transitions and yielded 8.91 MB raw, 4.94 MB Brotli — not re-measured on that artifact since. On the eight-language v8 container, `data/akshar.model` (§2, §10.2) is now itself pruned at the same threshold before shipping (necessary to hold the container under the 25 MB budget stated in §2), so `make web-model`’s own pass over it is close to a no-op (99.8% of the already-pruned trigrams kept) and the browser artifact is dominated by the shared lexicon (§8.6) rather than the LM: 23.30 MB raw, 13.97 MB Brotli. This is roughly three times the single-language Brotli figure, because the lexicon — the thing that made every language’s accuracy usable at all (§2.3) — ships inside the browser container too and was not shrunk specially for it. A smaller, browser-specific lexicon cap is the lever if that number needs to come down further; it has not been built, because it trades directly against the accuracy numbers in §2.3 for whichever languages it would cap harder, and that trade has not been asked for yet.

Chapter 11

Training

Implemented in `src/bin/train/train.rs`. One command produces the container:

```
make train-quick    # reranker on 100k pairs, ~10 min
make train-mid      # reranker on 500k pairs, ~40 min  <- validation gate
make train-full     # reranker on all 3.59M, ~4 h
```

11.1 What `--reranker-pairs` does and does not control

It sizes the reranker’s training set only. The EM emission model, the Kneser-Ney LM and the corpus vocabulary always consume the full data regardless of which target is run:

Component	Data used, every run
EM emissions + KN LM	all 3,588,793 parallel pairs
Vocabulary	470,012 words from a 1.5 GB corpus (hapax pruned at $f < 3$)
Reranker sparse table	<code>--reranker-pairs</code>

This matters when interpreting a quick run: it exercises the EM and LM changes at full scale, but not the reranker’s batching.

11.2 The four phases

Phase 1 — EM. Ingest pairs, seed emissions from the aligner, run 12 EM iterations, build the Kneser-Ney LM. ~170 s.

Phase 2 — Vocabulary. Stream the running-text corpus, count word frequencies, prune below $f = 3$. ~33 s.

Phase 3 — Reranker. Decode each training pair, extract dense and sparse features, and train the sparse table by softmax cross-entropy over the candidate list with **AdaGrad** [Duchi, Hazan & Singer 2011]. The objective is candidate reranking in the sense of [Collins 2000]. Above 200,000 pairs this runs **chunked**: batches of 100,000 are decoded once and reused for all epochs, which is why a full run is ~4 h rather than ~18 h.

Phase 4 — Pack. Prune emission rows unreachable from the vocabulary, quantise the sparse table, and write the v5 container.

11.3 Learning rate

AdaGrad adapts per slot, so the global schedule only needs a gentle anneal: `lr` starts at 0.05 and decays to 10% of that across the whole run, spread evenly over the batches.

This replaced a schedule that compounded $0.8^{(\text{epochs}-1)}$ *per batch* on top of a per-batch 0.995 — a factor of 0.6368 per batch, reaching 4.4×10^{-9} by batch 36. A `train-full` run under that schedule trained effectively on the first ~500k pairs and no-opped the remaining ~3M, which is why adding data had never helped.

11.4 Held-out monitoring

4,000 pairs are held out from the tail of the corpus, decoded once, and scored after every batch (chunked) or epoch (non-chunked). The table written into the container is the **best by dev loss**, not necessarily the last.

This is not optional hygiene. The sparse table has $\sim 10^6$ parameters and no regularisation, so overfitting is the default failure mode — and before this, only training loss was reported, which cannot distinguish learning from memorisation.

Watch for: dev loss falling monotonically, and the final learning rate within an order of magnitude of the first.

Caution. `train` writes `data/akshar.model` unless `--out` is given, so a training run silently replaces the released container. Back it up, or always pass `--out`, before experimenting.

11.5 Data

Artefact	Contents
<code>data/aksharantar/train_devanagari.jsonl</code>	3.59M Roman/Devanagari pairs
<code>data/aksharantar/valid_devanagari.jsonl</code>	held-out validation split
<code>data/aksharantar/test_devanagari.jsonl</code>	4,101-case test split
<code>data/store/corpus_clean.txt</code>	1.5 GB of running text
<code>data/eval/test_multiref.jsonl</code>	14,410 loose romanizations of the test set

Held-out text never contributes to vocabulary counts or the EM model.

Chapter 12

Evaluation methodology

This chapter states what is measured, how, and under what assumptions, so that every number elsewhere in the manual can be checked or contested.

12.1 Data

Benchmark. The AI4Bharat *Aksharantar* collection [Madhani et al. 2023], a public corpus of Roman/Devanagari word pairs, is the source for all eight languages. `prepare_pairs` (make data-prepare) NFC-normalises native words, lowercases romans, deduplicates train by a 128-bit key over the pair, and — critically — drops any train pair whose native word also appears in valid or test, so a word the reranker is scored on never also trains it (§17.1, D20, records what happened before this existed). Output: `data/pairs/{train,valid,test}.jsonl`, one shared file per split carrying a `lang` field rather than one file per language, shuffled with a fixed seed so the trainer never again sees one language for a long unbroken run (§17.1, D19).

Split	Pairs	Use
<code>train.jsonl</code>	7,308,261	EM, language model, reranker, all 8 languages
<code>valid.jsonl</code>	30,565	reranker holdout, blend calibration (<code>calibrate_blend</code>)
<code>test.jsonl</code>	48,277	all multi-language accuracy in this manual

Per-language pair counts are not uniform — this matters for reading §2.3 and §12.8 correctly, because a language with little clean training text has a lower *generation* ceiling, not just a harder ranking problem:

Lang	hin	mar	nep	san	kok	mai	brx	doi
train	1,297,699	1,246,308	2,277,802	1,780,680	460,325	209,691	34,480	1,276
valid	6,357	7,646	2,804	3,398	3,502	3,790	3,068	0
test	10,112	12,190	4,101	5,302	5,042	5,449	4,081	2,000

Dogri has no `valid` cases at all — too little data for a third split — so its `gamma_lang` (§8.6) falls back to `gamma_auto`. It and Bodo are the two languages whose `in@50` generation ceiling (§2.3, §12.8) is visibly lower than the other six, which is what this table is for: a ranking problem is fixed by better features or more

reranker supervision, a generation problem needs more raw training text, and conflating the two would send effort at the wrong lever.

Each `test.jsonl` case carries the original Aksharantar source field, and `eval_langs` partitions it the same three ways the original single-language harness did:

Stratum	Content
AK-Freq, AK-Uni, Dakshina	native words — grouped as “native” throughout
AK-NEI, AK-NEF, Wikidata	named entities — grouped as “entity”
everything else	mined pairs, reported by <code>eval_langs --json</code> but not headlined

Native strata are the headline metric because they measure the intended task. Entity accuracy is reported alongside, in full (§2.3), never pooled into one “accuracy” without saying so.

The lexicon’s frequency text is a separate, smaller source from the same family: an IndicCorp v2 sample per language (~300 MB each, pinned and SHA-256-verified via `scripts/data-manifest.tsv`, make `data-fetch`), not the 113 GB full corpus — a deliberate scope decision (this manual’s minimalism constraint, §2) verified sufficient by the numbers in §2.3 before being accepted. `build_lexicon` (§8.6) counts words from it directly; Nepali also keeps the original 86M-token `corpus_clean.txt` via `--extra`.

Contamination control. `prepare_pairs`’s train/valid/test split (above) is the primary defence. A second, independent one guards the lexicon and corpus bigram tables specifically: `core::holdout::is_holdout` reserves 1 line in 200 of running text by a fixed hash, and both `build_lexicon` and `build_corpus_bigrams` skip those lines when counting, so the same sentences used for sentence-level evaluation (§9.3.1) never also inform the frequency tables scored against them. The engine is constructed fresh per evaluation run with an **empty user dictionary**: the adaptive-learning path (§9.3) is not exercised by `eval_langs`, `evaluate_aksharantar` or `evaluate`, because feeding it gold answers during evaluation would make every later occurrence trivially correct — §12.10 is the one harness that deliberately does feed it gold answers, because measuring exactly that effect is its purpose, and it is reported as a separate, clearly-labelled number for exactly this reason.

12.2 Metrics

Let N be the number of test cases, D_i^* the gold Devanagari string for case i , and $\hat{D}_i^{(1)}, \dots, \hat{D}_i^{(k)}$ the engine’s ranked output.

Top- k accuracy. Exact string match, the primary metric:

$$\text{Acc}@k = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left[D_i^* \in \{ \hat{D}_i^{(1)}, \dots, \hat{D}_i^{(k)} \} \right].$$

Exact match is strict — a single wrong matra scores zero — and it is the right metric for an IME, where the user either gets the word or has to fix it. $k = 1$ measures the top suggestion; $k = 5$ approximates a visible candidate bar. Comparison is on NFC-normalised Unicode strings with no case folding. `eval_langs` (§12.6) additionally reports `top-3` and `in-list@k` — `Acc@k` under a different name, kept because “does the user have to scroll” is the natural reading for an IME’s suggestion list, where $k = 8$ is what both the IBus and browser front ends actually request (the browser widget’s on-screen page defaults to 5 of those 8; §2). `--ceiling n` adds `in@n` at a much larger n (50 throughout this manual): the same `Acc@k` computed at a k no real UI would ever show, so it measures pure generation, separated from the ranking loss that dominates at UI-sized k (§12.8’s oracle numbers are the single-language predecessor of this same idea).

--json writes every number in this section, split by language and stratum, to keep a machine-checkable copy next to the prose one.

Lenient scoring. Reported next to strict, never instead of it: a prediction is also credited if it differs from gold only by a standard Devanagari orthographic variation a reader treats as the same word — nukta presence (कागज़/कागज), chandrabindu vs. anusvara, or a nasal consonant + virama vs. anusvara before a stop (सन्त/संत). `lenient_key` (`eval_langs.rs`) canonicalises both strings under NFD before comparing; §2.3 reports the delta it adds pooled, and it is consistently a few points, never the difference between a weak and a strong result.

Mean reciprocal rank. Sensitive to *where* in the list the answer falls, not just whether it is present:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}, \quad \text{rank}_i = \min\{j : \hat{D}_i^{(j)} = D_i^*\},$$

with $1/\text{rank}_i = 0$ when the gold answer is absent from the returned list.

Character error rate. A graded measure, so that near-misses are distinguished from nonsense:

$$\text{CER} = \frac{\sum_i \text{lev}(\hat{D}_i^{(1)}, D_i^*)}{\sum_i |D_i^*|},$$

with `lev` the Levenshtein distance [Levenshtein 1966] over Unicode scalar values.

Oracle@*k*. The accuracy a *perfect* reranker would achieve on the candidate list actually generated:

$$\text{Oracle}@k = \frac{1}{N} \sum_{i=1}^N \mathbb{1}\left[D_i^* \in \text{Cand}_i^{(k)}\right].$$

This separates the two failure modes that a single accuracy number confounds: $\text{Oracle}@k - \text{Acc}@1$ is **ranking** loss (generated, mis-ordered), and $1 - \text{Oracle}@k$ is **generation** loss (never produced at all). The distinction drives the entire roadmap (§13).

Multi-reference accuracy. Roman input is ambiguous, so a prediction can be correct without matching the single reference. `data/eval/test_multiref.jsonl` collects 14,410 alternative romanizations; scoring credits a match against any reference for the same input. Reported alongside strict accuracy, never instead of it.

12.3 Protocol

Configuration. Unless stated otherwise: beam width 64, rerank cascade depth 24, $\gamma = 0.3$, $k = 5$ requested, `data/akshar.model`, no user dictionary, single desktop CPU. Every ablation varies exactly one factor via a documented environment switch (§9.3) against this fixed baseline.

Determinism. The engine is deterministic — no sampling, no RNG on the inference path, and hash containers on the ordering path use a fixed-seed hasher. Repeated runs of `evaluate_aksharantar` reproduce identical counts. Reported figures are therefore single runs, not averages, and any difference between two runs is a real difference in code, model or configuration.

Latency. Wall-clock per `get_suggestions` call, averaged over all 4,101 cases after model load, measured inside the harness rather than by timing the process (which would include a ~1.5 s cold start). Latency is reported to three decimal places in ms but should be read as $\pm 10\%$: it is sensitive to machine load, and

several figures in this manual were taken while a training job was running — those are marked where they appear.

12.4 Statistical treatment

Confidence intervals. `evaluate` reports bootstrap percentile intervals [Efron 1979]: resample the N per-case outcomes with replacement B times ($B = 1000$ by default, seed 42, via `rand_chacha:ChaCha8Rng`; v1.2.0 replaced a hand-rolled `SplitMix64` with this crate without changing the protocol), recompute the statistic on each resample, and take the 2.5th and 97.5th percentiles.

Comparing two configurations. Independent confidence intervals are the *wrong* tool here: both systems see the same cases, so their errors are correlated and overlapping intervals do not imply no difference. Comparisons use **McNemar’s test** [McNemar 1947] on the paired outcomes. With

$$b_{01} = \#\{i : A \text{ wrong, } B \text{ right}\}, \quad b_{10} = \#\{i : A \text{ right, } B \text{ wrong}\},$$

the concordant cases carry no information about which system is better, and under H_0 the discordant ones split evenly. The two-sided exact p -value is

$$p = 2 \sum_{i=0}^{\min(b_{01}, b_{10})} \binom{n}{i} / 2^n, \quad n = b_{01} + b_{10},$$

clipped at 1. Both b_{01} and b_{10} are reported with every comparison, not just p , because their magnitudes show whether a change is a small net effect over many disagreements or a genuinely consistent one.

Significance is claimed at $\alpha = 0.05$. **No correction is applied for multiple comparisons**, so the ablation table’s borderline entries ($0.01 < p < 0.05$) should be read as suggestive rather than established.

Effect sizes. Reported in percentage points on the relevant stratum. One standard error on AK-Freq at $n = 2,108$ and $p \approx 0.82$ is $\sqrt{p(1-p)/n} \approx 0.84\text{pp}$, which is the yardstick used throughout for calling a difference “within noise”.

12.5 Threats to validity

Stated so a reader can weigh the results rather than take them on trust.

Single benchmark. All accuracy comes from one Devanagari test set (Aksharantar, 4,101 cases) with domain-skewed priors. Per-language strata and a second benchmark (Dakshina [Roark et al. 2020], on which the IndicXlit comparison figures are usually quoted) are not evaluated here.

Baseline comparability. IndicXlit numbers (80.25% native, 52.67% named-entity top-1) are quoted from [Madhani et al. 2023], **not re-measured** in this environment. They are cited for scale, and no claim of a controlled head-to-head is made.

Isolated words. The headline metric scores words with no sentence context, which is not how an IME is used. `evaluate_sentences` measures in-context accuracy on held-out running text and is the more realistic figure; it is reported less often here simply because it has changed less.

Vocabulary overlap. The frequency prior is built from a news-domain corpus and the test set is drawn from a related distribution, so the prior’s contribution (+5.64pp, §9.1) may not transfer to out-of-domain input.

Tuning on the test set. γ , beam width and cascade depth were selected by sweeping against this test split. Those choices are mildly optimistic; the validation split exists and should be used for them.

12.6 Harnesses

Command	Reports
<code>make eval</code>	top-1/top-5 per stratum (single-language AK-Freq/AK-NEI/AK-NEF harness)
<code>make eval-full</code>	bootstrap CIs, MRR, latency
<code>make eval-ime</code>	plain-language report (correct-first-time, visible-in-top-5, keystrokes-saved) plus machine JSON at docs/generated/eval.json for regenerating every number in this manual
<code>make eval-errors</code>	oracle curves, error taxonomy, CER, collision bound
<code>make eval-sentences</code>	in-context word accuracy on held-out sentences (<code>--ctx-mode off\ oracle\ predicted</code> ; needs <code>make ctx-model</code>)
<code>make eval-langs [SPLIT=] [MODEL=] [JSON=]</code>	§2.3's table: per-language, per-stratum top-1/in-list/MRR, <code>--lang-aware</code> , <code>--ceiling</code> , lenient scoring
<code>cargo run --release --bin eval_session -- [--draws N] [--lang-aware]</code>	§12.10: cold vs. learning-enabled session accuracy
<code>make data-fetch / make data-prepare</code>	pinned, checksummed dataset download; data/pairs/*.jsonl build (§12.1)
<code>make ablate</code>	per-component contribution
<code>cargo test --release</code>	unit tests plus the accuracy regression guard

12.7 The accuracy regression guard

tests/accuracy_regression.rs evaluates a 400-case AK-Freq sample on every cargo test, failing below 76% top-1 or 88% top-5.

It exists because a **30.79pp regression once shipped through 92 green unit tests**. Every component was individually correct; their *composition* was wrong. No amount of unit testing detects that.

12.8 Headroom, and where the errors are

From `make eval-errors` at beam 256:

	AK-Freq	All
engine top-1 (strict / multi-ref)	81.83% / 82.16%	61.89% / 62.23%
decoder oracle @2	89.8%	71.4%
decoder oracle @5	92.4%	78.1%
decoder oracle @50	94.3%	85.1%
matra-only share of misses	51.9%	37.2%
top-1 if the matra class were solved	91.03%	76.08%
CER (engine top-1)	3.90%	11.66%

Three consequences, which set the roadmap:

1. **The gap is ranking, not generation.** The gold answer is in the decoder’s top 50 for 94.3% of native cases but ranked first for 81.8%. 12.5pp are generated and then mis-ranked.
2. **Most of that is a binary decision.** Oracle@2 is 89.8%, so 8.0 of the 12.5 points are a choice between the top two candidates.
3. **Half the errors are vowel signs.** 51.9% of native misses are *matra-only*: prediction and gold agree after stripping vowel-length and nasal marks.

An error is classified *matra-only* if prediction and gold become identical after deleting all matras, anusvara, visarga and chandrabindu; *halant-only* by the same construction on viramas; and *substantive* otherwise.

12.9 The collision bound

`analyze_errors` computes the ceiling for **any** string-only system whose sole prior is corpus unigram frequency. Let $A(R)$ be the set of Devanagari words observed for Roman input R across train, valid and test, and f the corpus frequency:

$$\text{Acc}^* = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left[D_i^* = \arg \max_{D \in A(R_i)} f(D) \right] = 99.15\%.$$

Only 0.85% of cases are unwinnable this way (1.6% of Roman inputs map to more than one gold form). **The dataset is not the constraint**, and a 90% target is not near any intrinsic ceiling.

12.10 Cold vs. session accuracy

Every number in this manual up to here, and every published transliteration result we are aware of including IndicXlit’s own ([Madhani et al. 2023]), is **cold**: each test case is scored in isolation, with an empty user dictionary (§12.1), and every distinct word counts once regardless of how often it is actually typed. That is the right protocol for a number meant to be compared across systems and papers, and this manual keeps reporting it unchanged for that reason. It is not what typing with this engine feels like, for two separable reasons, and `eval_session` (`src/bin/evaluate/eval_session.rs`) measures both.

Reason one: real typing is not uniform over the vocabulary. Natural language is Zipfian; a handful of words account for most keystrokes. Sampling test cases **weighted by the lexicon’s own per-language frequency** (§8.6) rather than uniformly, with learning still switched off, moves pooled native top-1 from 60.69% (§2.3’s uniform number) to **73.37%** on the same model. Nothing changed about the model here — only which of its already-correct answers get counted more, because they would in fact be typed more.

Reason two: the engine remembers. `user_confirms` (§9.3) writes a confirmed word into the user trie at a score band (900,000, §9.1) that outranks the decoder outright, so a word corrected once is answered correctly on every later occurrence for as long as the session lasts. `eval_session` draws romanised words with replacement, frequency-weighted as above, replays them through **one** engine instance in order, and calls `user_confirms` whenever the top-1 answer is wrong — simulating a user who, one way or another (picking from the list, or typing the word outright), ends up with the right word and lets the engine learn it, exactly the path both shipped front ends wire to a pick. Result, pooled over all eight languages, 20,000 draws each, `data/akshar.model`:

	cold (1st encounter)	2nd encounter	3rd–5th	6th+	session (blended)
pooled top-1	73.37%	99.62%	99.53%	99.61%	97.28%

Accuracy on a word does not creep upward with more corrections; it jumps after the *first* one and stays there, because the user-trie band is a near-total override, not a graded nudge. “Session” blends cold and already-learned encounters in the same frequency-weighted proportion they would actually occur in, so it is the single number closest to what typing with this engine feels like over time.

Two things this is not. It is not a claim that the shipped system achieves 97.28% on the comparable, cross-paper metric — §2.3’s cold, uniform number is that metric, and remains what this manual leads with. And it is a ceiling, not a guarantee: the simulation assumes every miss is eventually corrected and that the correction is captured, which is optimistic relative to a user who gives up, mistypes the correction too, or whose front end does not wire the pick back to `user_confirms`. Read together, §2.3 and this section bound the same system from two honest directions: never worse than 73.37% pooled once real usage frequency is accounted for, and not to be oversold beyond 97.28% no matter how favourably learning is modelled.

Chapter 13

Ablations: what each component is worth

Every component here can be switched off at runtime, so this table is reproducible from the shipped binary rather than from patched builds.

13.1 Switches

Variable	Effect
AKSHAR_NO_TRIE_UNION=1	skip the trie-constrained decode pass
AKSHAR_TRIE_ONLY=1	skip the free lattice beam
AKSHAR_NO_SPARSE=1	drop the 2^{20} sparse reranker table
AKSHAR_NO_RERANK=1	rank by raw decoder score, skipping the rerank stage
AKSHAR_GAMMA=<f>	override the dense/heuristic blend
AKSHAR_NO_TRIGRAM=1	force the LM to back off to bigrams
AKSHAR_NO_VARIANTS=1	decode the raw query only
AKSHAR_BEAM=<n>	beam width (default 64)
AKSHAR_RERANK_DEPTH=<n>	cascade depth (default 24)
AKSHAR_CACHE_SIZE=<n>	suggestion-cache size (default 256)
AKSHAR_DATA_DIR=<dir> ~/.local/share/akshar-ime, then /usr/share/akshar-ime)	model directory override (default: data/, then
AKSHAR_USER_TRIE_BASE=<n>	learned-word base score (default 900,000)
AKSHAR_FUZZY_BASE=<n> 150,000 per edit distance)	fuzzy-match base score (default 600,000; penalty
AKSHAR_KN_FIXED_DISCOUNT=1	single-discount LM ablation for the §13.6 comparison

13.2 The rerank stage, built up from raw decoder order

Measured 2026-09-08 (previous model; kept as the honest decomposition — the 2026-09-10 retrain did not re-measure the raw-decoder baseline row). An earlier version of this table treated $\gamma = 0$ as “no reranking”. That was wrong: $\gamma = 0$ still applies the frequency heuristic, which *is* part of the rerank stage. Building the stage up from the generative ranking gives the honest decomposition:

Ranking	AK-Freq top-1	Δ
raw decoder order (emit + lm)	75.38%	—
+ frequency heuristic ($\gamma = 0$)	81.02%	+5.64
+ 29 dense features	81.93%	+0.91
+ 2^{20} sparse table (shipped)	81.83%	−0.10

Reranking is worth +6.45pp overall on the 2026-09-08 model (+5.93pp on the retrain) — the second-largest contribution in the system after the trigram LM. But **most of that value is the three-parameter heuristic**

$$h(D) = \text{emit} + 0.85 \text{ lm} - 0.75 \log(1 + f(D)),$$

whose entire content is a corpus frequency prior the generative model does not have. The 10^6 -parameter learned stage added +0.91pp on top of it on the 2026-09-08 model (+0.24pp on the retrain), and the sparse half of that contributed nothing on native words then ($p = 0.851$, §13.4; −0.48pp on the retrain, untested for significance).

Reproduce with AKSHAR_NO_RERANK=1, AKSHAR_GAMMA=0.0, AKSHAR_NO_SPARSE=1.

13.3 Contribution of each remaining component

Full system: **80.98%** on AK-Freq (retrained 2026-09-10 model; the 2026-09-08 model measured 81.83% — the rows below are re-measured on the current model via `make ablate`):

Component removed	top-1	Δ
whole rerank stage	75.05%	−5.93
trigram LM (bigram only)	74.95%	−6.03
dense + sparse (heuristic only)	80.74%	−0.24
trie-constrained pass	80.50%	−0.48
sparse table (2^{20})	80.50%	−0.48
corpus lexicon	80.98%	0.00
query variants	80.98%	0.00

Previous (2026-09-08) values for reference: whole rerank −6.45, trigram −3.89, dense+sparse −0.81, trie pass −0.66, sparse +0.09, lexicon and variants 0.00. The paired McNemar tables below are the 2026-09-08 measurements, kept as the significance record.

The **language model and the frequency prior carry this system**. Everything learned discriminatively is marginal by comparison.

13.4 Paired significance

Marginal deltas are not evidence on their own. McNemar’s test on paired outcomes:

Removing the sparse table ($\sim 10^6$ parameters, 1.00 MB, 8.8% of the container):

Split	n	full	ablated	Δ	w→r	r→w	p
AK-Freq	2108	81.83%	81.93%	+0.09	15	13	0.851
AK-NEF	817	31.21%	29.38%	-1.84	3	18	0.0015
AK-NEI	1176	47.79%	46.85%	-0.94	13	24	0.099
ALL	4101	61.98%	61.40%	-0.59	31	55	0.013

Removing the dense reranker ($\gamma = 0$):

Split	n	full	ablated	Δ	w→r	r→w	p
AK-Freq	2108	81.83%	81.02%	-0.81	15	32	0.019
AK-NEF	817	31.21%	30.23%	-0.98	15	23	0.256
AK-NEI	1176	47.79%	46.17%	-1.62	18	37	0.015
ALL	4101	61.98%	60.91%	-1.07	48	92	0.0003

Two conclusions:

- The **sparse table does nothing on native words** ($p = 0.851$; the +0.09 is noise). Its entire measurable value is named entities. A paper reporting only native accuracy cannot justify 10^6 parameters for it.
- The **dense reranker is real but small**: -1.07pp pooled at $p = 0.0003$.

They are not independent. “Remove both” reproduces “remove dense” *exactly* — identical accuracies and identical discordant counts on every split — because at $\gamma = 0$ the blend returns $-h(D)$ and the sparse contribution is computed and then discarded. Sparse acts only *through* dense. Reporting them as two independent contributions would be wrong.

13.5 Components that were removed

Three subsystems were measured, found to contribute nothing, and deleted.

Corpus-wide SymSpell. Indexed a romanization of the top 100k corpus words and scored hits at 850,000 — above the decoder band. Cost **-30.79pp** native top-1 (81.74% → 50.95%), NEI 47.62 → 25.68, NEF 31.21 → 20.81. Given a fair test (distances verified, per-edit penalty) it still cost 9.6pp at a competing band, and at any safe band produced results *byte-identical* to being switched off. It never contributed recall at any setting.

Corpus lexicon. Dead by construction: the shipped path never loaded one, and its data file was no longer produced by the pipeline. 0.00pp on every split.

Lattice CRF and pair model. 1,108 lines across two modules (`crf.rs` 702 + `pair_model.rs` 406, per git history), never constructed, never packed into the container. The “modified Kneser-Ney” work of an earlier commit had landed here — in modules that never shipped.

Total removed: ~1,452 lines, no measurable accuracy change.

13.6 Modified Kneser-Ney against a single discount

Two otherwise-identical 100k runs:

LM	AK-Freq	AK-NEF
modified KN ($D_1=0.605$, $D_2=1.036$, $D_3=1.444$)	81.55% / 92.31%	30.60% / 53.00%
fixed $\delta = 0.75$	81.26% / 92.22%	30.84% / 53.86%

+0.29pp is inside one standard error (0.84pp at $n = 2108$), and fixed δ is ahead on AK-NEF. **Modified Kneser-Ney is now correctly implemented but is not measurably better than a single discount on this data.** It is retained because it is the standard estimator and free at runtime; it should not be described as an improvement.

Chapter 14

Performance

14.1 Where the time goes

Profiled over 1,000 real queries (`examples/profile_decode.rs`, mean input length 10 characters):

Phase	ms/query	share
free lattice beam	0.55	67%
trie-constrained beam	0.12	15%
decode_union (both + merge)	0.686	84%
reranker	0.047	6%
engine overhead	0.082	10%
end to end	0.816	

make eval-full reports **0.63–0.82 ms/query** at $k = 10$ over the full test set, run to run (0.75 ms on the 2026-09-08 re-verification run; the phase breakdown above sums to 0.816 ms on the profiling run — same budget, different runs). Cold start is ~1.5 s (1.50 s wall for a 5-case process on 2026-09-08).

14.2 How it got there

The engine was at 3.5 ms. Three changes, none of which cost accuracy:

Binary-search LM lookups (5.4x). bigram_weight and trigram_weight scanned their successor rows linearly. The beam expands ~5,000 hypotheses per step with one LM lookup each — roughly 50,000 linear scans per query. Rows are stored ascending by id, so this is a binary search. Free beam: 3.128 → 0.575 ms. Output was byte-identical (1944/2108 before and after).

O(n) beam pruning. select_nth_unstable_by instead of a full sort to keep the best 64 of ~5,000.

Lazy arena materialisation. Path cells are written only for hypotheses that survive pruning, cutting arena writes from $O(\text{beam} \times \text{edges})$ to $O(\text{beam})$ per step.

Indexed akshara lookup. TranslitModel::akshara_id was a linear scan over the vocabulary, called ~500,000 times while building the word trie at start-up. Cold start: 4.35 → 1.55 s.

The lesson worth carrying: the reranker was assumed to be the bottleneck and was 6% of the time. **Profile before optimising.**

14.3 Trade-offs available

Configuration	native top-1	latency
default (beam 64, both passes)	81.83%	0.67–0.82 ms
trie-constrained pass only	66.18%	0.18 ms

Trie-only decoding is 4x faster but costs 15.65pp: the corpus vocabulary does not cover the test set. Beam width and cascade depth are tunable via AKSHAR_BEAM and AKSHAR_RERANK_DEPTH.

Chapter 15

Deployment

15.1 Linux / IBus

```
make release && sudo make install && make restart-ibus
```

src/ibus_engine.c handles key events and the candidate UI, calling the Rust core through the C ABI in src/c_api.rs. The library is installed to /usr/lib, the engine to /usr/lib/ibus/engines, the component XML to /usr/share/ibus/component, and the model to /usr/share/akshar-ime.

Learned state persists to the user's data directory (src/persistence.rs); make reset-learning clears it.

15.2 Browser / WebAssembly

```
make wasm           # builds packages/engine-wasm/pkg/
make web-model      # builds data/akshar_wasm.model (browser profile)
```

```
import { AksharIME } from 'packages/engine-js/akshar-ime.js';
await AksharIME.init({ modelUrl: './models/akshar_wasm.model' });
AksharIME.attach(document.querySelector('input'));
```

The wrapper (packages/engine-js/akshar-ime.js) calls createEngine(modelUrl, lexiconUrl, rerankerUrl) from packages/engine-wasm/pkg/akshar_ime.js (raw factory createEngineFromModelUrl — see src/wasm.rs); its engine object exposes getSuggestions, confirm, export_state/import_state.

The runnable playground (editor UI) ships from the separate private repo Fundamentals-Work/akshar-playground — self-contained (engine, wrapper and model bundled under public/, no CDN, no external requests), deployable to Cloudflare Workers or Pages. Sync flow: copy packages/engine-js/, packages/engine-wasm/pkg/ and data/akshar_wasm.model into its public/ tree (paths in that repo's README), commit, redeploy.

Serve akshar_wasm.model with Content-Encoding: br and a long cache lifetime; it is 4.94 MB compressed and immutable.

WasmEngine::from_bytes(model, lexicon, weights) retains its lexicon parameter as an ignored no-op so existing callers keep working. Pass null.

Browser latency has not been re-measured since 2026-09-06; the last figure was ~3–7 ms per call, taken before the 4.3x decoder speed-up, so it should be expected to be substantially better.

15.3 Offline tools

Binary	Purpose
<code>train</code>	full pipeline, writes the container
<code>pack_model</code> , <code>repack_model</code>	assemble / re-encode a container
<code>prune_lm</code> , <code>prune_model</code> , <code>quantize_model</code>	browser-profile compaction
<code>build_wordfreq_text</code>	vocabulary from running text
<code>romanize</code>	Devanagari → Roman with orthographic canonicalisation
<code>probe_model</code>	inspect container sections, or decode one word
<code>evaluate</code> , <code>evaluate_aksharantar</code> , <code>evaluate_sentences</code>	accuracy harnesses
<code>analyze_errors</code>	oracle curves, taxonomy, collision bound

`src/fuzzy/grammar.rs` supports `romanize` and `evaluate_sentences` and is not part of the runtime engine.

Chapter 16

Source map

15,773 lines of Rust (`find src -name '*.rs' | xargs wc -l`), plus `src/ibus_engine.c`. Runtime core first, then tooling. Counts re-measured 2026-09-22; earlier editions of this table drifted and are not trusted (a standing risk this table cannot fix itself out of — treat any copy of it older than its own date stamp the same way).

File	Lines	Role
<code>core/engine.rs</code>	1339	orchestration, candidate fusion, language selection, learning
<code>core/em_trainer.rs</code>	1049	EM (scaled forward-backward), Kneser-Ney LM construction
<code>core/reranker.rs</code>	948	dense + sparse features, WordCounts, language conditioning, blend, cascade
<code>core/codec.rs</code>	819	compact container encoding
<code>core/unified.rs</code>	712	container layout, v1–v8 version dispatch
<code>core/decoder.rs</code>	657	lattice beam search, Dictionary trait, both passes
<code>fuzzy/grammar.rs</code>	525	orthographic canonicalisation (offline tools only)
<code>wasm.rs</code>	423	WebAssembly bindings
<code>core/translit_model.rs</code>	406	model access: emissions, LM lookups, backoff, end-of-word
<code>core/lexicon.rs</code>	333	§8.6: the shared multi-language word-knowledge automaton
<code>core/normalizer.rs</code>	311	query-variant rewriting
<code>core/alignment.rs</code>	250	deterministic aligner used to seed EM
<code>core/akshara.rs</code>	239	akshara segmentation
<code>core/trie.rs</code>	210	user-learned dictionary
<code>core/context.rs</code>	208	user bigram re-ranking
<code>c_api.rs</code>	197	C ABI for IBus
<code>core/wordtrie.rs</code>	109	legacy single-corpus vocabulary trie (pre-v8 containers)

File	Lines	Role
core/reranker_weights.rs	114	compiled-in dense weight defaults
fuzzy/sympell.rs	113	symmetric-delete index
core/mod.rs	101	module map, ablation env-flag registry
learning.rs	123	learning orchestration
persistence.rs	79	learned-state serialisation
core/holdout.rs	66	held-out split helper (§12.1)
bin/train/train.rs	1169	training pipeline, all 8 languages, lexicon-aware
bin/evaluate/*	3041	eval_langs.rs (425), eval_session.rs (249, §12.10), analyze_errors.rs, evaluate.rs, eval_ime.rs, evaluate_sentences.rs, tune_weights.rs, evaluate_aksharantar.rs
bin/build/*	2193	prepare_pairs.rs (246, §12.1), build_lexicon.rs (228, §8.6), calibrate_blend.rs (233), prune_lm.rs (165, §10.4), plus container/vocab tooling

Tests: tests/accuracy_regression.rs, tests/fuzzy_behavior.rs, plus 107 module tests (cargo test --release --lib) as of this table. Diagnostics: examples/profile_decode.rs, examples/ablate_paired.rs, examples/diag_reranker.rs, examples/diag_fuzzy.rs.

Chapter 17

Known defects and limitations

Recorded openly. Nothing here is hidden in a footnote.

17.1 Defects resolved in v1.2.0

- **D18 — user fuzzy path unreachable (Resolved):** The user fuzzy band was previously placed at 50,000 (38,000 at distance 1), below the decoder’s 8th candidate (~250,000). Calibrated to `DE-FAULT_FUZZY_BASE = 600_000` with penalty `150_000 * d`. Exact decodes (at 800,000) remain strictly protected, while typos of confirmed words now recover at rank 2–4. The regression test `learned_word_should_be_recoverable_from_a_typo` is unignored and passes 100%.
- **Trigram lower-order estimate (Resolved, §6.5):** Corrected from highest-order bigram to textbook continuation count $N_{1+}(\bullet, b, c)$ via `bigram_cont_right`.
- **End-of-word symbol `</w>` (Resolved, §6.5):** Interned and appended to all training words during LM counting, closing trigram probability mass word-finally.
- **W_DENSE trap / joint training (Resolved, §18.2):** Unified model container v6 now carries trained `dense_weights`. `train.rs` fits dense and sparse weights jointly using sample-level gradient accumulation with warm-start from `W_DENSE`.

Defects resolved in the multi-language overhaul (Unreleased, this manual’s own revision date). The three below are why the reranker had never measured as beating the three-parameter heuristic (§8.1) despite being individually correct code — the same “each component right, the composition wrong” pattern D18’s regression came from, three more times over, and the direct cause of the old “language-agnostic” framing this manual corrected in §2:

- **D19 — reranker trained on one language (Resolved).** Aksharantar’s training file is sorted by language. `--reranker-pairs 100000` (the `train-quick/train-mid` default used for every prior measurement in this manual) therefore trained on Hindi and nothing else, however many languages the EM/LM ingested. `prepare_pairs` (§12.1) now shuffles with a fixed seed before any pair count is taken.
- **D20 — EM/LM memorised the reranker’s own evaluation pairs (Resolved).** Nothing partitioned train from the reranker’s held-out set at the *word* level, only at the pair-count level, so words the reranker was later scored on had usually already been seen, verbatim, by the EM alignment and the language model it ranks against. `prepare_pairs` now drops any train pair whose native word also appears in valid or test (§12.1).
- **D21 — dense-feature normalisation mismatch (Resolved).** `dense_mean/ dense_std` (§8.2, v5) were computed once and never refreshed after a retrain shifted the underlying `emit/lm` distributions by 15–25%, so `score_dense` was standardising against statistics that no longer matched the features being fed in. `train.rs` now recomputes both every run via Welford’s online algorithm.

Two further defects, found while auditing the decoder and EM aligner for the same overhaul, are unrelated to the three above but share their “quietly wrong for a specific input class” shape:

- **D22 — end-of-word signal trained but never spent (Resolved).** The LM’s `</w>` transition (§6.5) was interned and counted during training, but no decode-time call ever charged its cost, and container packing pruned it out of shipped models as an apparently-unused table. `TranslitModel::end_weight` (§6) now applies it in the decoder’s completion step. +1.2pp macro native top-1 (Sanskrit +3.6pp) on the eight-language validation set.
- **D23 — Sanskrit avagraha misclassified as phonetically emitting (Resolved).** U+093D (avagraha) and U+0900 (inverted chandrabindu) were segmented as their own akshara units, so the EM aligner tried to align them against Roman characters that do not represent any sound — corrupting alignment for 51,000+ Sanskrit training pairs. Both are now classified as combining marks that attach to the preceding syllable (§4.3).
- **D24 — beam search mixed candidate lengths (Resolved).** The beam kept the best `beam_width` hypotheses across the *whole* lattice at each step, not per input position, so a short candidate spelling and a long one competed directly for the same beam slots although they had consumed different amounts of Roman input — silently biasing survival toward short spellings regardless of their score. §7.2’s beam is now position-synchronous (`stacks: Vec<Vec<Hyp>>` indexed by Roman byte position). +4.6pp macro in-list@8 with no latency cost; the beam width was in fact reduced from 64 to 32 (§2) afterward with no further accuracy loss, since the old beam had been wasting slots on paths that could never be compared fairly anyway.

17.2 Open limitations

The candidate-union score space is hand-tuned u64 bands. Two of the three serious defects found in this system originated there. While D18 has been calibrated safely, full log-linear fusion remains the architectural goal.

17.3 Things that do not work as their names suggest

The discriminative reranker standalone ($\gamma = 1.0$) does not beat the heuristic alone. Even with joint dense+sparse fitting (v1.2.0), the learned model peaks at $\gamma \approx 0.2\text{--}0.3$ (+0.28pp over heuristic on AK-Freq). As proved in §18.2, the limit is the feature representation itself (global counts cannot resolve the 75.9% in-beam matra ranking ties), confirming that the discriminative stage must be supplemented by a Factored Matra Model rather than more linear weights.

Modified Kneser-Ney is not measurably better than a single discount (§13.6).

Query-variant rewriting contributes 0.00pp (§9.1). 311 lines of Dijkstra over a rewrite transducer whose only decoded output is the identity variant. Retained because it is the only cold-start mechanism for spelling alternation, but it is not currently earning its place.

17.4 Scope limitations

- **Resolved this revision: script-general engine, single-language priors.** Every prior edition of this manual through 2026-09-10 described the modelling core as Devanagari-script-general while conceding the shipped *priors* (vocabulary, suffix list) were Nepali-only in practice — so every other Devanagari language’s accuracy was capped by a frequency table built from a different language’s corpus, however correct its phonetics were. §2 and §8.6 describe the fix (one lexicon automaton, eight languages’ frequencies, per-language ranking); §2.3 has the current per-language numbers. This bullet is kept, struck through in spirit rather than deleted, because a publication citing an earlier snapshot of this system should not have to guess which claim changed.

- **Installed as one keyboard per language.** `devanagari-smart.xml` now declares nine IBus engine names — a language-blind `devanagari-smart` plus `devanagari-smart-{hi,ne,mr,sa,kok,mai,brx,doi}`, each calling `set_language` (§8.6, §9) on `focus_in`. This closes the previous limitation (a single `<language>ne</language>` tag regardless of what the model actually served) but is untested beyond the C engine’s own build (§15.1); no manual OS-layer verification across all nine has been logged here yet.
- **One dataset family.** Aksharantar, for training and all headline numbers. The Dakshina benchmark, on which IndicXlit’s own headline is usually quoted, is still not evaluated here (§12.5).
- **Named entities are well behind native-word accuracy in every language** (§2.3): pooled entity top-1 is 31.59% against 60.69% native, and the gap is not closing with more languages — it is a feature gap (§8.2’s dense features are shape/frequency signals that do not model “is this plausibly a name”), not a data one.
- **Bodo and Dogri are generation-limited, not just ranking-limited** (§12.1, §12.8): their `in@50` ceiling is visibly below the other six languages’, so more reranker supervision cannot close their gap alone — they need more clean training text before ranking improvements matter.
- **The eight-language browser profile has not been measured for latency** (§10.4 has its size, not its query cost); only the single-language figure in §2 predates this overhaul.
- **--reranker-pairs 0 (the full ~7.3M-pair analogue of the old train-full) has not been retried since D19–D21 were fixed.** It was tested and falsified under the broken pipeline (§18.1); whether more supervision helps once training data is actually multi-language and leak-free is an open question, not a re-confirmed negative.

Chapter 18

Roadmap

The goal is **Devanagari-script accuracy**: one engine that serves every language written in Devanagari lipi, judged by a pooled script headline with per-language strata underneath — not by tuning to any single language’s distribution. Delivery surfaces (Cloudflare playground, Chrome extension) follow accuracy; they do not lead it. The live work plan is docs/plans/devanagari-script-plan.md.

The measured decomposition of the 18.2 missing points on native words:

$$\underbrace{94.3 - 81.8 = 12.5}_{\text{ranking: generated, mis-ranked}} + \underbrace{100 - 94.3 = 5.7}_{\text{generation: never in top 50}}$$

Work is ordered by expected points per unit of effort.

- 1. Factored matra model.** 51.9% of native misses are matra-only, and solving that class alone reaches 91.03%. This is the largest single lever, and it is a better-posed problem than whole-word reranking: predict vowel-sign assignment given a consonant skeleton.
- 2. A top-2 discriminator.** Oracle@2 is 89.8%, so 8.0 of the 12.5 ranking points are a binary choice between the top two candidates. Training a model specifically on that decision is a smaller and better-conditioned problem than a 50-way ranker.
- 3. Log-linear candidate fusion.** Replace the u64 bands with a single log-linear score in which every source contributes a feature. This removes an entire defect class — both the 30.8pp regression and D18 were band-ordering bugs — and makes the sources jointly tunable.
- 4. End-of-word symbol.** Cheap, and it targets matra errors, which concentrate word-finally.
- 5. Refit the dense weights.** See §18.2 — this replaced “retrain the reranker on more data”, which was tested and falsified.
- 6. Continuation-count trigram backoff.** Textbook correctness; modest gain.

18.1 Result: more supervision was tested and does not help

The plan above once contained “retrain the reranker on all 3.59M pairs”, on the theory that a 2^{20} -parameter table trained on 100k examples was starved. That was run and **falsified**, and the negative result is more useful than the hypothesis was.

train-mid (500,000 reranker pairs — 5x the released model, with the corrected learning-rate schedule and held-out early stopping) produced:

	AK-Freq	AK-NEF	AK-NEI
released model (100k pairs)	81.83%	31.21%	47.79%
train-mid (500k pairs)	81.59%	30.97%	48.30%

Five times the ranking supervision changed nothing measurable. Held-out dev loss during that run never once beat having no sparse table at all:

after	dev loss	dev top-1
no table (start)	1.8582	54.18%
batch 1	1.8785	47.23%
batch 2	1.8719	49.07%
batch 3	1.9192	47.82%
batch 4	1.9208	48.08%
batch 5	1.9182	48.67%

Per-batch training loss on *fresh* data climbed ($1.58 \rightarrow 1.90 \rightarrow 2.46$), which is the generalisation gap stated directly. The learning rate annealed exactly as designed (0.05×0.631^n : 0.0500, 0.0315, 0.0199, 0.0126, 0.0079), so this is not a schedule problem.

And the γ sweep — the stated falsification test — came back unmoved:

γ	released model	train-mid
0.0 (heuristic only)	81.02%	80.83%
0.3 (shipped)	81.83%	81.59%
0.5	—	81.45%
0.7	—	79.46%
1.0 (learned only)	76.47%	74.05%

18.2 Resolution: Joint Dense + Sparse Training (Landed in v1.2.0)

In v1.2.0, the pipeline was updated to jointly train the 29 dense weights and the 2^{20} sparse table under the softmax cross-entropy objective:

1. **v6 Unified Container:** Added `dense_weights: Vec<f64>` to `UnifiedModel`, allowing trained dense parameters to travel directly with the container to inference.
2. **Sample-Level Gradient Accumulation:** Fixed a critical mathematical defect where dense weight updates were previously computed per-candidate (50 gradient steps per sample), destroying Ada-Grad’s learning rate adaptation. Gradients are now accumulated across all candidates for a sample ($\mathbf{g} = \sum_i (p_i - y_i) \mathbf{z}_i$) and applied once per sample.
3. **Warm-Start & Normalization Alignment:** `dense_weights` is warm-started from `w_DENSE` to prevent initial divergence, and the model preserves the exact `MEAN_DENSE` and `STD_DENSE` coordinates used during feature extraction.

18.2.1 Measured Results on train-mid and train-full (v1.2.0)

- **train-mid (500k pairs, 5 epochs):**

- Dev loss trajectory: dropped monotonically from 1.8513 $\rightarrow$ **1.0422** (dev top-1: 54.58% $\rightarrow$ **67.98%**).
- AK-Freq: **80.88%** top-1 (81.02% at $\gamma = 0.1$, +0.28pp over baseline).
- AK-NEF: **29.87%** top-1 (+0.74pp over baseline).
- AK-NEI: **45.92%** top-1 (+0.17pp over baseline).
- **train-full (3.59M pairs, 36 batches, 5 epochs, 58m runtime):**
 - Total samples processed: 15,938,635 samples across all 3.59M parallel pairs.
 - Training loss: converged from 1.0954 down to **0.7120**.
 - Quantized sparse table: 35,810 non-zero slots (3.42%), clip 3.04913, scale 41.6512.
 - AK-Freq: **81.02%** top-1 (top-5: **91.75%**).
 - AK-NEI: **45.75%** top-1 (top-5: **70.24%**).
 - AK-NEF: **29.50%** top-1 (top-5: **51.90%**).
 - γ sweep: $\gamma = 0.0 \rightarrow 80.74\%$, $\gamma = 0.1 \rightarrow 80.88\%$, $\gamma = 0.2 \rightarrow$ **81.02%**, $\gamma = 0.3 \rightarrow$ **81.02%**, $\gamma = 0.4 \rightarrow 80.65\%$, $\gamma = 1.0 \rightarrow 70.78\%$.

18.2.2 Falsification Test Finding

With joint dense+sparse fitting landed, γ still peaks around 0.1–0.3. As formulated in the falsification criteria below, **this decisively confirms that global count features are the limit, not the fitting.**

Error forensics (make eval-errors) on the remaining AK-Freq misses show that **75.9% of misses already have the gold word in the top-10 candidates** (and oracle@50 stands at 94.3%). The misses are predominantly fine-grained matra alternations (e.g., short vs. long *i/u*, halant vs. schwa), which global word-level counts cannot distinguish.

The remaining roadmap therefore prioritises the **Factored Matra Model** (§12.5) over further linear discriminative tuning.

Chapter 19

Experimental record

Final numbers cannot show what was tried and rejected. The working documents in docs/plans/archive/ preserve that record; this chapter summarises it.

The accuracy figures in the archive are historical — measured before the 2026-09-06 defect fixes — and do not describe the shipped system. This manual is authoritative for current numbers.

19.1 Approaches evaluated and rejected

Approach	Why rejected
IndicXlit transformer (~11M params) as the core	~40 MB; breaks the browser budget by an order of magnitude. Retained only as an offline reference point.
NADIR-style non-autoregressive neural decoder	~50 MB; same constraint.
Neural character LM interpolated with the KN LM	5–10 MB for < 1pp on the tail.
Corpus word-bigram context table	19.5 MB of container for +0.16pp. Removed in container v4.
Corpus-wide fuzzy matching	Cost 30.79pp of native top-1 and contributed no recall at any score band (§13.5). Removed.
Corpus roman→devanagari lexicon	0.00pp on every stratum; dead by construction. Removed.
Lattice CRF over the decode graph	Half-built, never wired in, 702 lines. Removed rather than left as dead weight; recoverable from git.
Joint pair-model over (akshara, chunk) states	Built by the trainer but never packed or loaded. Removed.
Modified Kneser-Ney over a single discount	Implemented correctly, but +0.29pp is inside one standard error (§13.6). Retained as the standard estimator, not claimed as an improvement.
More reranker supervision (5x)	Tested at 500k pairs; changed nothing, and dev loss never beat having no sparse table (§13.2).

19.2 Approaches considered but not implemented

Recorded in docs/plans/archive/2026-09-05-research-agenda.md:

- **Context-tree weighting** [Willems, Shtarkov & Tjalkens 1995] as a parameter-free alternative to Kneser-Ney smoothing.
- **A* anytime decoding** [Hart, Nilsson & Raphael 1968] over the lattice, for exact search with a latency budget.
- **An entropy harness** to measure the information budget — how many bits the Roman input actually carries about the Devanagari output — and thus bound what any model can achieve.
- **Incremental decoding**: caching the beam per prefix and extending it by one character, rather than re-decoding the whole growing prefix on each keystroke.

19.3 Archive index

Document	Records
2026-08-01-generative-transliteration-design.md	Original design: the source-channel decision, akshara units, first results.
2026-09-03-transliteration-accuracy-research.md	Error analysis and a ranked technique shortlist (E0–E7) with expected gains.
2026-09-03-accuracy-experiments.md	The experiment log : E0–E3 with measured deltas, the WFST core, depth-2 pair context.
2026-09-05-data-flow.md	How raw text becomes the artefacts a keystroke touches.
2026-09-05-data-research.md	Literature review: IndicXlit’s data usage, context in production IMEs [Kirov et al. 2024], larger corpora.
2026-09-05-research-agenda.md	Mathematics considered but not executed.
2026-09-05-roadmap-to-90.md	First plan to 90%: audit of how every byte of data is used.
2026-09-05-path-past-90.md	Its revision, with W0 measurement-gate results.
2026-09-05-mathematics.md	Complete mathematical treatment (equations with code references).
2026-09-06-repair-and-path-to-90.md	Measured defect audit, landed fixes, improvement plan (A/B shipped, C–F open).

Chapter 20

References

Work this system builds on, grouped by where it is used. Section numbers point to the chapter that relies on it.

20.1 Model and training

Baum, L. E., Petrie, T., Soules, G. and Weiss, N. (1970). *A Maximization Technique Occurring in the Statistical Analysis of Probabilistic Functions of Markov Chains*. *Annals of Mathematical Statistics* 41(1), 164–171. — the forward-backward recursions (§5.3).

Dempster, A. P., Laird, N. M. and Rubin, D. B. (1977). *Maximum Likelihood from Incomplete Data via the EM Algorithm*. *JRSS B* 39(1), 1–38. — the EM framework (§5).

Rabiner, L. R. (1989). *A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition*. *Proceedings of the IEEE* 77(2), 257–286. — scaling of the forward-backward recursions, §V.A (§5.3).

Li, H., Zhang, M. and Su, J. (2004). *A Joint Source-Channel Model for Machine Transliteration*. *ACL*. — the source-channel formulation this system uses (§4.2).

Shannon, C. E. (1948). *A Mathematical Theory of Communication*. *Bell System Technical Journal*. — the noisy-channel decomposition (§4.2).

20.2 Language modelling

Ney, H., Essen, U. and Kneser, R. (1994). *On Structuring Probabilistic Dependences in Stochastic Language Modelling*. *Computer Speech & Language* 8(1), 1–38. — absolute discounting (§6.2).

Kneser, R. and Ney, H. (1995). *Improved Backing-off for M-gram Language Modeling*. *ICASSP*. — continuation probabilities (§6.1).

Chen, S. F. and Goodman, J. (1999). *An Empirical Study of Smoothing Techniques for Language Modeling*. *Computer Speech & Language* 13(4), 359–394. — modified Kneser-Ney, and the constraint $0 \leq D_i \leq i$ this system had been violating (§6.2).

Stolcke, A. (1998). *Entropy-based Pruning of Backoff Language Models*. *DARPA Broadcast News Transcription and Understanding Workshop*. — the browser profile’s LM pruning (§8.4).

20.3 Decoding

Viterbi, A. J. (1967). *Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm*. IEEE Trans. Information Theory.

Lowerre, B. (1976). *The HARPY Speech Recognition System*. PhD thesis, CMU. — beam search (§7.2).

Mohri, M. (1997). *Finite-State Transducers in Language and Speech Processing*. Computational Linguistics 23(2), 269–311. — the tropical semiring and lattice formulation (§7.1).

Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Trans. SSC. — A* anytime decoding, considered but not implemented (archive: research agenda).

Malik, M. G. A., Boitet, C. and Bhattacharyya, P. (2008). *Hindi Urdu Machine Transliteration using Finite-State Transducers*. COLING. — prior art for FST-based statistical transliteration on this script pair (§2.1).

Rajan, V. (2014). *Konkanverter — A Finite State Transducer based Statistical Machine Transliteration Engine for Konkani Language*. Proceedings of the Fifth Workshop on South and Southeast Asian NLP, COLING. — prior art for FST transliteration specifically involving Devanagari and, notably, one of this system’s own eight languages (§2.1).

20.4 Discriminative reranking

Collins, M. (2000). *Discriminative Reranking for Natural Language Parsing*. ICML. — the reranking-over-k-best formulation (§8).

Och, F. J. (2003). *Minimum Error Rate Training in Statistical Machine Translation*. ACL. — MERT, used by the legacy fallback reranker.

Weinberger, K. et al. (2009). *Feature Hashing for Large Scale Multitask Learning*. ICML. — the hashing trick behind the sparse table (§8.3).

Duchi, J., Hazan, E. and Singer, Y. (2011). *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*. JMLR 12, 2121–2159. — AdaGrad (§11.3).

Daumé III, H. (2007). *Frustratingly Easy Domain Adaptation*. ACL. — the shared/language-tagged feature duplication and language dropout §8.6 uses to condition ranking on eight languages without training eight rerankers.

Sannigrahi, S. and Bawden, R. (2023). *Investigating Lexical Sharing in Multilingual Machine Translation for Indian Languages*. EAMT. arXiv:2305.03207. — related-but-different prior art for sharing lexical resources across Indian languages, in translation rather than transliteration decoding and ranking (§2.1, §8.6).

20.5 Strings, structures and statistics

Levenshtein, V. I. (1966). *Binary Codes Capable of Correcting Deletions, Insertions and Reversals*. Soviet Physics Doklady 10(8), 707–710.

Damerau, F. J. (1964). *A Technique for Computer Detection and Correction of Spelling Errors*. CACM 7(3), 171–176.

Garbe, W. *SymSpell: 1000x faster spelling correction*. <https://github.com/wolfgangarbe/SymSpell> — symmetric-delete indexing (§7.4 of the engine chapter).

Fredkin, E. (1960). *Trie Memory*. CACM 3(9), 490–499.

Welford, B. P. (1962). *Note on a Method for Calculating Corrected Sums of Squares and Products*. *Technometrics* 4(3), 419–420. — the online statistics used for dense-feature normalisation.

Witten, I. H., Moffat, A. and Bell, T. C. (1999). *Managing Gigabytes*, 2nd ed. Morgan Kaufmann. — front coding and varint dictionary compression (§8.3).

McNemar, Q. (1947). *Note on the sampling error of the difference between correlated proportions or percentages*. *Psychometrika* 12(2), 153–157. — the paired significance test used for every ablation (§13.4).

Efron, B. (1979). *Bootstrap Methods: Another Look at the Jackknife*. *Annals of Statistics* 7(1), 1–26. — the confidence intervals reported by `evaluate`.

Willems, F. M. J., Shtarkov, Y. M. and Tjalkens, T. J. (1995). *The Context-Tree Weighting Method: Basic Properties*. *IEEE Trans. Information Theory* 41(3), 653–664. — considered as a parameter-free alternative to Kneser-Ney; not implemented (§13).

20.6 Data, benchmarks and writing systems

Madhani, Y., Parthan, S., Bedekar, P. et al. (2023). *Aksharantar: Open Indic-language Transliteration Datasets and Models for the Next Billion Users*. Findings of EMNLP. <https://aclanthology.org/2023.findings-emnlp.4/> — the training and test data, and the IndicXlit baseline this manual compares against.

Roark, B., Wolf-Sonkin, L., Kirov, C., Mielke, S. J., Johnny, C., Demirsahin, I. and Hall, K. (2020). *Processing South Asian Languages Written in the Latin Script: the Dakshina Dataset*. LREC. — the benchmark this system does **not** evaluate on (§12.1).

Kirov, C. et al. (2024). *Context-aware Transliteration of Romanized South Asian Languages*. *Computational Linguistics* 50(2), 475–534. — title corrected 2026-09-22 (a prior revision of this manual had it as “...for Input Method Editors,” which is not this paper’s title). Sentence-level context from mono-script text, not the session-level, confirmation-driven context this manual adds in §9.3 and evaluates in §12.10 — a different axis, not prior art for it as far as we have checked.

Daniels, P. T. (1990). *Fundamentals of Grammatology*. *JAOS* 110(4), 727–731. — the *abugida* class to which Devanagari belongs (§4.3).

The Unicode Consortium. *The Unicode Standard*, Chapter 12: South and Central Asia-I. — Devanagari encoding, virama behaviour, ZWJ/ZWNJ semantics (§4.3).

20.7 Implementation

Steele, G. L., Lea, D. and Flood, C. H. (2014). *Fast Splittable Pseudorandom Number Generators*. OOPSLA. — the `splitmix64` finaliser used for path hashing.

Chapter 21

Glossary

Every term this manual uses in a technical sense. Where a term is due to a particular author, the reference is given.

Abugida — a writing system in which each consonant carries an inherent vowel that a diacritic modifies or suppresses [Daniels 1990]. Devanagari is one; this is why characters are the wrong modelling unit and aksharas are the right one.

Akshara — the orthographic syllable of Brahmic scripts, and the unit this system models. Formally (consonant halanta)* consonant? (matra | independent vowel) (nasal | visarga)*.

Anusvara (ँ) — a diacritic marking nasalisation. Frequently omitted or inserted inconsistently in Roman input, and a common source of matra-only errors.

Backoff — in an n -gram model, falling back to a shorter context when the full context was unseen, paying a weight $-\log \lambda$ for doing so.

Beam search — approximate search that keeps only the b best partial hypotheses at each step [Lowerre 1976]. Here $b = 64$ by default.

Bootstrap confidence interval — an interval obtained by resampling the observed per-case outcomes with replacement and taking percentiles of the resulting statistic [Efron 1979].

CER (character error rate) — total Levenshtein distance between prediction and gold, divided by total gold length. A graded alternative to exact match.

Chandrabindu (ं) — a nasalisation diacritic distinct from anusvara.

Chunk — a contiguous run of 1–5 Roman characters emitted by one akshara in the shipped model (training alignments additionally permit the empty string).

Codebook quantisation — storing weights as 8-bit indices into a 256-entry table of f32 values, rather than as full floats.

Collision bound — the accuracy ceiling for any system whose only prior is corpus unigram frequency; 99.15% here. Distinguishes “the model is weak” from “the task is ambiguous”.

Conjunct — two or more consonants joined by a halanta into one visual and orthographic unit, e.g. क्ष. Segmented as a single akshara.

Continuation probability — in Kneser-Ney smoothing, a lower-order estimate based on the number of *distinct contexts* a token appears in rather than its raw frequency [Kneser & Ney 1995]. The reason *Kong* is a poor guess in a novel context despite being frequent.

CSR (compressed sparse row) — storing a ragged 2-D structure as one flat value array plus row offsets.

Damerau-Levenshtein distance — edit distance allowing insertion, deletion, substitution and transposition [Damerau 1964; Levenshtein 1966].

Delta varint encoding — storing an ascending integer sequence as variable-length gaps rather than absolute values. Requires and preserves sorted order, which is also what makes binary search valid at runtime.

Discount (D_i) — the mass subtracted from an n -gram’s count before normalising, and redistributed to unseen events. Modified Kneser-Ney uses three, selected by count, each constrained to $0 \leq D_i \leq i$ [Chen & Goodman 1999].

Emission — $P(\text{chunk} \mid \text{akshara})$, the channel model learned by EM.

EM (expectation-maximisation) — iterative maximum-likelihood estimation with latent variables [Dempster, Laird & Rubin 1977]; here the latent variable is the alignment between Roman chunks and aksharas.

Forward-backward — the dynamic program computing posterior occupancy in a chain model [Baum et al. 1970]; the E-step of EM here.

Front coding — dictionary compression storing each entry as a shared-prefix length plus a suffix.

Halanta / virama (◌ँ) — the vowel-killer diacritic. Suppresses a consonant’s inherent vowel and binds it to the next consonant.

Hashing trick — mapping an unbounded feature space into a fixed-size table by hashing, accepting collisions in exchange for a bounded memory budget [Weinberger et al. 2009].

Inherent vowel / schwa — the vowel a bare Devanagari consonant carries (क = ka , not k). Written or omitted at the typist’s discretion, which is a major source of alignment ambiguity.

Lattice — a DAG whose nodes are input positions and whose edges are labelled hypotheses. Decoding is shortest-path over it.

Matra — a dependent vowel sign attached to a consonant (◌ा, ◌ि, ◌ी, ...). **51.9% of this system’s native errors are matra-only.**

McNemar’s test — a paired significance test for two classifiers on the same cases, using only the discordant pairs [McNemar 1947].

MRR (mean reciprocal rank) — the mean of $1/\text{rank}$ of the gold answer, 0 when absent. Sensitive to position, not just presence.

Multi-reference — scoring against any of several acceptable romanizations, rather than a single reference.

NFC — Unicode Normalization Form C (canonical composition). All string comparison here is on NFC-normalised text.

Oracle@ k — the accuracy a perfect reranker would reach on the candidates actually generated. Separates ranking loss from generation loss.

Purnabiram (।) — the Devanagari full stop.

Semiring, tropical — $(\min, +)$ arithmetic on negative log probabilities [Mohri 1997]. Converts maximum-probability search into shortest-path search, and is why all weights in this system add.

Source-channel model — factoring $P(D \mid R) \propto P(R \mid D)P(D)$ into a channel and a source [Shannon 1948]; applied to transliteration by [Li, Zhang & Su 2004].

SymSpell — spelling correction by precomputed deletion variants, giving lookup independent of dictionary size [Garbe]. A delete-set match is a *necessary* condition only, so results must be distance-verified.

Top- k accuracy — the fraction of cases whose gold string appears in the first k suggestions, by exact match.

Trie — a prefix tree [Fredkin 1960]. Used for the user dictionary and for the vocabulary-constrained decode pass.

Virama — see *halanta*.

Visarga (◌ः) — a diacritic representing a final voiceless breath.

Welford's algorithm — numerically stable online computation of mean and variance [Welford 1962].

ZWJ / ZWNJ — zero-width joiner and non-joiner (U+200D, U+200C). Preserved next to viramas so eyelash-ra ळ keeps its form.